PREMIUM EDITION

.NET INTERVIEW ROADMAP

SENIOR-STYLE ANSWERS FOR 0 – 2.5 YEARS CANDIDATES

PREPARED BY

Rutik Pimpale

.NET 8 · C# 12 · ASP.NET Core · EF Core · Web API · Clean Architecture

NAVIGATION

Table of Contents

CLICK ANY ENTRY TO JUMP TO THE SECTION

15 TOPIC 15: CACHING

- Q1: What caching strategies do you use in real .NET applications?
- Q2: IMemoryCache vs IDistributedCache (Redis) — when do you use which?

16 TOPIC 16: BASIC SYSTEM DESIGN THINKING

- Q1: How would you design a small e-commerce backend in .NET? ★

17 TOPIC 17: SQL BASICS

- Q1: Explain the different SQL Joins. ★
- Q2: What is Database Normalization?
- Q3: What is Indexing and how does it affect performance?

18 TOPIC 18: PROJECT STRUCTURE & CLEAN ARCHITECTURE

- Q1: How do you structure a real-world ASP.NET Core project (Clean Architecture basics)? ★

19 QUICK REVISION CHEAT SHEET

```
      </ul>
    </div>
<div class="toc-card">
      <a class="toc-topic" href="#interview-tips">
        <span class="toc-num">20</span>
        <span class="toc-topic-title">INTERVIEW
TIPS</span>
      </a>
      <ul class="toc-qs">

      </ul>
    </div>
```

TOPIC 1: C# ESSENTIALS

Q1: What is the difference between Value Types and Reference Types in C#? (IMPORTANT)

This is one of the most fundamental things in C# and the source of a lot of bugs in real projects, so I want to explain it the way I actually think about it.

A **value type** stores the **actual data** inside the variable itself. It's small, lightweight, and usually lives on the **stack** (or inline inside its containing object). When you assign one value type to another, you get a **completely independent copy**. Examples: `int`, `bool`, `double`, `char`, `struct`, `enum`, `DateTime`.

A **reference type** stores a **pointer** to the actual object that lives on the **heap**. When you assign one reference variable to another, both variables point to the **same object** in memory. Examples: `class`, `string`, `array`, `interface`, `delegate`, `record` (when declared as class).

Why it matters in real projects: I once spent half a day debugging an issue where I "copied" a `List<Customer>` and modifications to the copy were also showing up in the original. That's because `List<T>` is a reference type — both variables were pointing to the same list. The fix was a proper deep copy.

- **Use struct (value type)** for **small, immutable data** that's frequently created — like `Point`, `Money`, `DateRange`. Less GC pressure.
- **Use class (reference type)** for everything else, especially anything with identity, behavior, or that you want to share across the app.

- Treating `string` like a value type. It's a **reference type** but feels like a value type because it's **immutable**.
- Defining large structs (>16 bytes). Every assignment copies the whole thing — slower than a class.
- Forgetting that `struct` has a default parameterless constructor that zeroes out everything — you can't prevent it.

Boxing & Unboxing (very common follow-up): when you stuff a value type into an `object` or interface, .NET wraps it in a heap object — that's **boxing**. Pulling it back out is **unboxing**. Both are slow and create GC pressure. Avoid them in hot loops. Generics (`List<int>` instead of `ArrayList`) were created largely to eliminate boxing.

```
// Reference type — both variables point to same list
var listA = new List<int> { 1, 2, 3 };
var listB = listA;
listB.Add(4);
Console.WriteLine(listA.Count); // 4 — surprised? That's reference semantics.

// Value type — independent copies
int a = 10;
int b = a;
b = 20;
Console.WriteLine(a); // 10 — untouched.

// Boxing in action — avoid in hot paths
object boxed = 42;          // int → object: boxed (heap allocation)
int back  = (int)boxed;     // unbox — costly cast
```

- *Where exactly is a `struct` stored if it's a field inside a `class`?* — On the **heap**, inline inside the containing class. The "stack vs heap" rule is really "depends where the variable lives."
- *Why is `string` immutable in .NET?* — Thread safety, security (paths/connection strings can't be mutated mid-flight), interning for memory savings, and safe use as dictionary keys.

Q2: What is the difference between var, dynamic, and object?

They look similar but they're three completely different things, and mixing them up is a sign of a junior developer.

- `var` — purely a **compile-time** convenience. The compiler infers the exact type from the right-hand side. After that, the variable has a strict, fixed type. **Zero runtime cost.**
- `object` — the **base type of everything** in .NET. Holds anything, but to use it as something specific you must **cast**. Storing value types in `object` causes **boxing**.
- `dynamic` — the type is resolved **at runtime** by the **DLR (Dynamic Language Runtime)**. The compiler does **no checking**. Slower, no IntelliSense, errors only show up when the line actually runs.

- `var` everywhere for local variables — keeps code clean: `var users = await _db.Users.ToListAsync();` is much better than `List<User> users = ...`.
- `object` only when I genuinely need to hold "anything," like a generic event payload.
- `dynamic` almost never. Maybe for COM interop, ExpandoObject, or working with truly schema-less JSON. **It's a last resort.**

- Using `dynamic` "to make life easier" when binding to JSON. Better: define a DTO or use `JsonElement`.
- Using `object` as method parameter type. You lose type safety. Use generics instead (`T`).
- Reading `var` as "weak typing." It's not — `var x = "hi"; x = 5;` is a compile error.

Performance note: `dynamic` calls go through the DLR and are roughly **10–100× slower** than direct calls. Never put `dynamic` inside a hot loop.

```
var name = "Rutik";       // compiler sees "string" → fixed
// name = 10;             // ❌ compile error — already string

object obj = "Hello";
// int len = obj.Length;  // ❌ obj is "object" — needs cast
int len = ((string)obj).Length;

dynamic d = "Hello";
Console.WriteLine(d.Length);   // ✅ works at runtime
d = 10;
Console.WriteLine(d.Length);   // ✅ COMPILES, ❌ crashes at runtime
```

- *Can `var` be used for fields or method return types?* — No. Only for local variables where the compiler can see the assigned value.
- *What's the difference between `dynamic` and `object`?* — `object` requires explicit casts to call members; `dynamic` skips compile-time checks entirely. `object` is type-safe; `dynamic` is not.

Q3: What is the difference between string and StringBuilder?

A: The short answer: `string` is **immutable**, `StringBuilder` is **mutable**. The deeper answer is about understanding **memory and GC pressure**.

Every time you "modify" a string — `s += "x"`, `s.Replace(...)`, `s.ToUpper()` — you're actually creating a **brand new string object** on the heap. The old one becomes garbage. In a tight loop, this is a **disaster** for performance.

`StringBuilder` keeps an **internal mutable buffer** that grows as needed. You append into the same buffer — no new allocations per append.

- **Use** `string` **and** `+` when you have a **small, fixed number of concatenations** — the C# compiler often optimizes these into a single `string.Concat` call anyway.
- **Use** `StringBuilder` inside **loops**, when building large strings, or when you don't know the final size.
- **Use** `string.Join` **or** `string.Concat` when you already have a collection — they're as fast as `StringBuilder` and cleaner.
- **Use** `string.Create` **or** `Span<char>` for the absolute hottest paths (advanced).

Real-world example: I had to generate a 50,000-row CSV export. The first version used `result += line + "\n"` in a loop and took **8 seconds**. Switched to `StringBuilder` — dropped to **180 ms**. Same logic, ~40× faster, just because I stopped allocating tens of thousands of garbage strings.

- Using `StringBuilder` for tiny fixed strings — the overhead of creating it is more than what you save.
- Forgetting to set initial capacity for known-size builds: `new StringBuilder(capacity: 4096)` avoids re-allocations.
- Using `string.Format` in tight loops — it's slower than `StringBuilder.AppendFormat` or interpolation.

```
// ❌ Bad — 1000 garbage strings
string result = "";
for (int i = 0; i < 1000; i++)
    result += i + ",";

// ✅ Good — single buffer
var sb = new StringBuilder(capacity: 4096);
for (int i = 0; i < 1000; i++)
    sb.Append(i).Append(',');
var result2 = sb.ToString();

// ✅ Even better when you have a collection
var ids = Enumerable.Range(1, 1000);
var csv = string.Join(",", ids);
```

- *How does string interning help memory?* — Identical string literals share a single instance in the **intern pool**, saving memory. Useful for repeated tokens.
- *When would* `Span<char>` *or* `stackalloc` *beat* `StringBuilder`*?* — For small, short-lived buffers in performance-critical code where you want **zero heap allocations**.

Q4: What is the difference between ref, out, in, and params?

A: These four keywords change **how arguments are passed** to a method. Most freshers only know `ref` and `out`, but a senior should know all four and **why each exists**.

Keyword	Direction	Must be initialized before call	Method must assign	Use Case
`ref`	In + Out	✅ Yes	❌ Optional	Modify caller's variable
`out`	Out only	❌ No	✅ Yes (before return)	Return multiple values; `TryParse` pattern
`in`	In only (read-only ref)	✅ Yes	❌ No (cannot modify)	Pass large structs by reference, read-only
`params`	In	—	—	Variable number of arguments

- `ref` — "I want the method to read AND modify my variable." Rarely needed; usually a sign you should return an object instead.
- `out` — Classic use is the `TryXxx` pattern: `int.TryParse(s, out int n)`. With C# 7+ you can declare it inline, which is much cleaner.
- `in` — Performance optimization. Avoids copying large structs while guaranteeing the method can't change them. **Freshers almost never know this exists** — it's a great differentiator in interviews.
- `params` — Convenience for variadic args like `Console.WriteLine(string format, params object[] args)`.
- Using `ref` / `out` when a return value or tuple would be cleaner. C# 7 tuples kill most use cases for `out`.
- Using `in` on small structs — the indirection cost is more than the copy cost. Only use for structs **larger than ~16 bytes**.
- Mutating a captured variable inside `params` and expecting it to update the caller — `params` is just an array, it doesn't pass by reference.

```
// out — TryParse pattern (very common)
if (int.TryParse(input, out var number))
    Console.WriteLine($"Got {number}");

// ref — modify caller's value
void Increment(ref int x) => x++;
int n = 5;
Increment(ref n);
Console.WriteLine(n); // 6

// in — read-only reference (perf trick for big structs)
struct BigData { public long A, B, C, D, E, F; }
void Process(in BigData data)
{
    Console.WriteLine(data.A);
    // data.A = 1; // ❌ compile error — read-only
}

// params — variadic args
int Sum(params int[] nums) => nums.Sum();
Sum(1, 2, 3, 4);
```

- *Why prefer tuples over `out` parameters in modern C#?* — Tuples are cleaner, work nicely with deconstruction (`var (ok, value) = TryGet();`), and don't force the caller to declare a variable upfront.
- *Can you use `ref` with reference types?* — Yes, but it means "I want to **change which object the caller's variable points to**," not "modify the object." Different and rarely needed.

TOPIC 2: OOP CONCEPTS

Q1: What are the 4 Pillars of OOP — explained with real examples? (IMPORTANT)

A: Every interviewer asks this, and most candidates parrot textbook definitions. The way to stand out is to explain them with **examples from a real codebase**, not "a Car has wheels."

Pillar	Real meaning	Real-world example I've seen
Encapsulation	Hide internals; expose only what's safe to use	A `BankAccount` class exposes `Deposit()` / `Withdraw()` but the `_balance` field is private — no one can mutate it directly
Inheritance	Reuse common behavior in a base class	`BaseEntity` with `Id`, `CreatedOn`, `IsDeleted` — every entity inherits it
Polymorphism	Same method, different behavior depending on type	`INotificationSender` with `EmailSender`, `SmsSender`, `PushSender` — service code just calls `.Send()` and the right one runs
Abstraction	Hide complexity behind a simple interface	A controller depends on `IPaymentService` and doesn't care if it's Stripe, Razorpay, or a mock

- **Encapsulation = hiding data.** ("how" things are stored)
- **Abstraction = hiding complexity.** ("how" things are done)

People mix these up constantly in interviews. The cleanest distinction: encapsulation is about **access control** (private/public); abstraction is about **simplifying the public interface** (interfaces, abstract classes).

these aren't academic ideas. Without them you get classes that:

- Expose mutable internal state → bugs that are impossible to trace.
- Duplicate the same audit code in 50 entities → nightmare to maintain.
- Hardcode `new StripeService()` everywhere → impossible to test.
- Public auto-properties everywhere (`public string Name { get; set; }`) — that's not encapsulation, that's a struct in disguise. Use private setters or `init` for true encapsulation.
- Inheritance for code reuse when **composition** would be cleaner. (See SOLID — Liskov.)
- Confusing polymorphism with method overloading. **Overloading = compile-time polymorphism**, **overriding = runtime polymorphism**. Both count, but interviewers usually mean the second one.

```
// Encapsulation — internal state is protected
public class BankAccount
{
    private decimal _balance;
    public decimal Balance => _balance;          // read-only outside
    public void Deposit(decimal amount)
    {
        if (amount <= 0) throw new ArgumentException("Must be positive");
        _balance += amount;
    }
}

// Abstraction + Polymorphism
public interface INotificationSender
{
    Task SendAsync(string to, string message);
}

public class EmailSender : INotificationSender { /* SMTP */ public Task SendAsync(string t, string m) =>
Task.CompletedTask; }
public class SmsSender   : INotificationSender { /* Twilio */ public Task SendAsync(string t, string m) =>
Task.CompletedTask; }

// Caller doesn't know or care which one runs
public class AlertService
{
    private readonly INotificationSender _sender;
    public AlertService(INotificationSender sender) => _sender = sender;
    public Task Notify(string user, string msg) => _sender.SendAsync(user, msg);
}
```

- *Is C# a "purely" object-oriented language?* — No. It has primitive value types, supports functional features (LINQ, lambdas, records, pattern matching), and even some procedural code (top-level statements).
- *Can you have polymorphism without inheritance?* — Yes — through **interfaces**. In modern C#, that's actually the preferred way; favor interface-based polymorphism over deep class hierarchies.

Q2: Abstract Class vs Interface — when do you use which? (IMPORTANT)

Both let you define a contract, but they answer different questions:

- **Abstract class** answers *"what is this thing?"* (an `Animal`, an `Employee`)
- **Interface** answers *"what can this thing do?"* (`IComparable`, `IDisposable`, `IFlyable`)

Feature	Abstract Class	Interface
Implementation	Can have full method bodies	Only signatures (default methods since C# 8)
Fields	Yes	No (only properties)
Constructor	Yes	No
Inheritance	Single (one base class)	Multiple
Access modifiers	Any	Public by default (modern C# allows others)

> *Default to **interfaces**. Use an abstract class only when you have **shared implementation** that multiple subclasses*

> *genuinely need.*

Why? Because interfaces give you:

- **Mockability** for unit tests.
- **Multiple implementations** — easy to swap.
- **No accidental coupling** to a base class hierarchy.

Use an abstract class when you have a real — common skeleton, with specific steps overridden by children.

In one of my projects:

- `BaseRepository<T>` is an **abstract class** — handles common CRUD with EF Core, leaves entity-specific filtering to children.
- `IAuditable`, `ISoftDeletable`, `IEmailSender` are **interfaces** — capabilities a class can opt into.
- Choosing abstract class "in case I need shared code later." YAGNI — start with an interface and refactor if needed.
- Putting business logic in an abstract base class — makes testing the children harder because you can't isolate them.
- Forgetting that since C# 8, interfaces can have **default methods**, so the gap between abstract classes and interfaces has narrowed.

```
// Abstract class — shared implementation + forced override
public abstract class Animal
{
    public string Name { get; init; }
    public abstract void Speak();                    // children must implement
    public void Sleep() => Console.WriteLine($"{Name} is sleeping");  // shared
}

// Interface — pure contract / capability
public interface IFlyable
{
    void Fly();
}

public class Bird : Animal, IFlyable
{
    public override void Speak() => Console.WriteLine("Tweet");
    public void Fly() => Console.WriteLine($"{Name} is flying");
}
```

- *When would default interface methods (C# 8+) be a good fit?* — When you need to **add a method to an existing interface without breaking implementers** — common in library design. Avoid them in app code; they confuse the inheritance picture.
- *Can an abstract class have no abstract methods?* — Yes. You'd do that to **prevent instantiation** while still providing shared code. But if you have no abstract methods, consider whether `sealed` + static helpers is cleaner.

Q3: virtual vs override vs new — what's the difference?

These three keywords control how methods behave when inherited, and getting them wrong leads to silent, hard-to-find bugs.

- `virtual` — base class says "this method **can** be overridden by children."
- `override` — child class says "I'm **replacing** the base behavior — proper polymorphism."
- `new` — child class says "I'm **hiding** the base method with a different one." This is **rarely what you want** — it breaks polymorphism.

Why this matters: with `override`, the **runtime type** decides which method runs. With `new`, the **declared (compile-time) type** decides — meaning a `Base b = new Child();` will silently call the wrong method.

Real-world example: I once inherited a codebase where someone had used `new` to "override" `Save()` in `AuditRepository`. Worked fine in tests where the variable was typed `AuditRepository`. In production, the DI container injected it as `IRepository`, and audit logging silently stopped working for **months**. Switching to `virtual + override` fixed it instantly.

- Using `new` to silence a compiler warning ("X hides inherited member"). The warning is telling you to use `override`. Stop and think.
- Marking everything `virtual` "just in case" — overusing it makes the API harder to reason about and slows JIT optimization.
- Forgetting that C# methods are **non-virtual by default** (opposite of Java).

```
public class Base
{
    public virtual void Show() => Console.WriteLine("Base");
}

public class ChildOverride : Base
{
    public override void Show() => Console.WriteLine("Child Override");
}

public class ChildNew : Base
{
    public new void Show() => Console.WriteLine("Child New");
}

Base a = new ChildOverride();
a.Show();   // "Child Override" — polymorphism works ✅

Base b = new ChildNew();
b.Show();   // "Base" — `new` silently breaks polymorphism ❌
```

- *Can a `private` method be virtual?* — No. `virtual` requires the method to be visible to derived classes.
- *What does `sealed override` do?* — Overrides a virtual method **and** prevents further overrides down the chain. Useful when you want to lock in behavior at a specific level.

TOPIC 3: SOLID PRINCIPLES

Q1: Explain SOLID principles with real-world examples. (IMPORTANT)

A: SOLID is five design principles that, when followed, make code **easier to change, easier to test, and harder to break**. I'll go through each with an example I've actually used.

S — Single Responsibility Principle (SRP) A class should have **one reason to change**.

I once saw a `UserService` class that handled DB calls, email sending, password hashing, and PDF generation. Every change broke something else. Splitting it into `UserRepository`, `EmailService`, `PasswordHasher`, and `UserReportGenerator` made each one small, testable, and predictable.

O — Open/Closed Principle (OCP) Open for **extension**, closed for **modification**.

When we needed a third payment provider, we shouldn't have to edit `PaymentService`. Instead, `IPaymentProvider` is the contract; `StripeProvider`, `RazorpayProvider`, `PayPalProvider` are implementations. New provider = new class, zero changes to existing code.

L — Liskov Substitution Principle (LSP) A child class should be usable wherever the parent is used, **without breaking behavior**.

Classic violation: `Square` inheriting from `Rectangle` and overriding `SetWidth` to also set height. Code that worked with `Rectangle` breaks for `Square`. The fix: don't model "Square is a Rectangle" with inheritance — they should both implement `IShape`.

Don't force classes to implement methods they don't need.

If `IRepository` has 15 methods but most repositories only use 4, split it: `IReadRepository`, `IWriteRepository`, `IPagedRepository`. Classes implement only what they need, and you avoid `throw new NotImplementedException()` everywhere — which I've seen kill production.

D — Dependency Inversion Principle (DIP) Depend on **abstractions**, not concrete classes.

Controllers should depend on `IUserService`, not `UserService`. That's literally what ASP.NET Core's built-in DI container is built around. Without DIP, you can't unit test, you can't swap implementations, and you can't run in different environments cleanly.

Real impact in my projects: SOLID isn't theoretical. The codebases I've worked in that **followed** SOLID had unit tests that ran in seconds, refactors that took hours instead of weeks, and onboarded new devs in days. The ones that didn't were tar pits.

- Treating SOLID as a checklist instead of guidelines — over-applying SRP leads to **class explosion** (10 classes for what should be 1).
- Creating interfaces "for everything" even when there's only one implementation. Not always wrong, but often premature abstraction.
- Confusing DI (the technique) with DIP (the principle). DI is **how** you implement DIP.

```
// OCP + DIP in action
public interface IPaymentProvider
{
    Task<PaymentResult> ChargeAsync(decimal amount, string token);
}

public class StripeProvider   : IPaymentProvider { /* ... */ public Task<PaymentResult> ChargeAsync(decimal a,
string t) => Task.FromResult(new PaymentResult()); }
public class RazorpayProvider : IPaymentProvider { /* ... */ public Task<PaymentResult> ChargeAsync(decimal a,
string t) => Task.FromResult(new PaymentResult()); }

public class CheckoutService
{
    private readonly IPaymentProvider _payment;
    public CheckoutService(IPaymentProvider payment) => _payment = payment;

    public Task<PaymentResult> Pay(decimal amount, string token)
        => _payment.ChargeAsync(amount, token);
}

public record PaymentResult(bool Success = true);
```

- *Which SOLID principle do you think is the most important?* — In my experience, **SRP and DIP**. SRP keeps classes maintainable; DIP keeps the architecture testable. If you do those two well, the others tend to fall into place.
- *Can SOLID be over-applied?* — Absolutely. Over-applying SRP or ISP creates a maze of tiny classes and interfaces. Apply principles **when complexity demands it**, not preemptively.

TOPIC 4: .NET FRAMEWORK VS MODERN .NET

Q1: What's the difference between .NET Framework, .NET Core, and .NET 6/7/8? (IMPORTANT)

A: This is a question about **history and direction**, and you should be able to give a confident, structured answer.

Aspect	.NET Framework	.NET Core	.NET 5 / 6 / 7 / 8+
First released	2002	2016	2020 onwards
Platform	Windows only	Cross-platform	Cross-platform
Open source	No	Yes	Yes
Performance	Good	Better	Best — improves every release
Future	Maintenance only — **dead-ended at 4.8**	Replaced by .NET 5+	**The only .NET going forward**
Use for	Legacy WCF, WebForms, old enterprise	Cross-platform apps before 2020	All new development

The story in one paragraph: .NET Framework was Windows-only and tied to the OS. Microsoft built **.NET Core** as a fresh, cross-platform, open-source rewrite. With **.NET 5** they merged the two product lines into a single unified ".NET" — that's why we don't say "Core" anymore. **.NET 6** was the first LTS of the unified era, **.NET 8** is the current LTS, and each version brings huge perf wins.

(this is what separates a senior answer):

- **.NET 6 (LTS)** — Minimal APIs, hot reload, unified `Program.cs`.
- **.NET 7** — Performance focus; rate limiting middleware; native AOT preview.
- **.NET 8 (LTS)** — Full **Native AOT** support for Web APIs (faster cold start, smaller container images), **Keyed DI services**, `IExceptionHandler` for clean global error handling, `TimeProvider` for testable time, big perf gains across the board.

Real-world example: I migrated a .NET Framework 4.8 reporting service to .NET 8. Docker image size dropped from **~280 MB to ~110 MB**, p95 latency improved by **~30%**, and we could finally deploy on the company's existing Linux Kubernetes cluster instead of dedicated Windows VMs. Just changing runtime gave us measurable wins without changing business logic.

- Saying ".NET Core 5" or ".NET Core 8" — **wrong**. After 2020 it's just .NET 5/6/7/8.
- Picking .NET Framework for a new project — there's basically no reason to in 2026.
- Confusing **.NET Standard** (a spec for shared libraries) with **.NET runtimes**.

- *What's an LTS release?* — Long-Term Support, supported ~3 years. .NET 6 and 8 are LTS; 7 was non-LTS (18-month support). Production apps usually target LTS.
- *Is migration from .NET Framework to .NET 8 always smooth?* — No. WCF, WebForms, AppDomains, Remoting, and old third-party libraries are common blockers. Microsoft's **.NET Upgrade Assistant** helps but isn't magic.

TOPIC 5: ASP.NET CORE & MIDDLEWARE

Q1: What is ASP.NET Core and why is it the standard for new projects?

A: ASP.NET Core is Microsoft's modern, **cross-platform, open-source, high-performance** framework for building web APIs, web apps, and real-time services. It's a complete rewrite of the old ASP.NET, designed for the cloud era.

What makes it stand out (and these are the talking points I use in interviews):

- **Cross-platform** — runs on Windows, Linux, macOS, Docker, Kubernetes.
- **Kestrel server** — one of the fastest web servers in the world; consistently top of TechEmpower benchmarks.
- **Built-in DI** — no need for Autofac/Ninject for most apps.
- **Unified MVC + Web API + Razor + SignalR** — single programming model.
- **Modular pipeline** — middleware composition gives a lot of control.
- **Cloud-native** — small images, fast cold start, environment-based configuration, health checks built in.
- **First-class async** — every modern API supports `async/await`.

every backend I've built in the last 3 years is ASP.NET Core. A typical microservice: Web API + EF Core + JWT + Serilog + Docker, deployed to Kubernetes on Linux. The same codebase runs on my Windows laptop and on a Linux pod — zero changes.

- Maintaining legacy systems that depend on **WCF, WebForms, or AppDomains** — those don't exist in ASP.NET Core.
- Tightly Windows-specific apps that use Windows-only COM/DCOM heavily.
- Mixing up old ASP.NET (Windows + IIS) with ASP.NET Core (cross-platform, Kestrel). Don't do this in interviews.
- Treating it like classic ASP.NET MVC and looking for `Global.asax`, `web.config`, etc. — gone. Everything happens in `Program.cs` now.

```
// Modern minimal Program.cs (.NET 6+)
var builder = WebApplication.CreateBuilder(args);

builder.Services.AddControllers();
builder.Services.AddDbContext<AppDbContext>(o ⇒
    o.UseSqlServer(builder.Configuration.GetConnectionString("Default")));
builder.Services.AddScoped<IUserService, UserService>();

var app = builder.Build();

app.UseExceptionHandler("/error");
app.UseHttpsRedirection();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

- *Why does ASP.NET Core need a reverse proxy in production?* — Kestrel is fast but lacks features production ops needs — request buffering, advanced TLS config, header manipulation. **Nginx, IIS, or YARP** in front handles that.
- *What's the difference between Razor Pages, MVC, and Web API in ASP.NET Core?* — Razor Pages = page-based server rendering (simple sites), MVC = controller/view server rendering, Web API = JSON for SPAs/mobile. All three live in the same framework and can coexist.

Q2: Explain the ASP.NET Core Middleware Pipeline. (IMPORTANT)

A: The middleware pipeline is the **heart of ASP.NET Core**. Every HTTP request flows through a chain of middleware components, each of which can:

1. **Inspect or modify** the request,
2. **Pass it to the next** middleware (`await next()`), or **short-circuit** and respond directly,
3. **Inspect or modify** the response on the way back.

Mental model I use: think of middleware like . The request goes inward layer by layer, and the response comes back out through the same layers in reverse.

Order matters — a lot. The order you register middleware in `Program.cs` is the order they execute. Get it wrong and you get subtle bugs that work in dev but fail in prod.

1. `UseExceptionHandler` — catch errors from everything below.
2. `UseHsts` / `UseHttpsRedirection` — enforce HTTPS.
3. `UseStaticFiles` — serve `wwwroot` content.
4. `UseRouting` — match URL to endpoint.
5. `UseCors` — handle cross-origin.
6. `UseAuthentication` — figure out who the user is.
7. `UseAuthorization` — decide if they're allowed.
8. `UseEndpoints` / `MapControllers` — execute the endpoint.

> *Putting `UseAuthorization` **before** `UseAuthentication`. Authorization will run before the user is identified, so `User.Identity.IsAuthenticated` is always `false` and everything 401s. Authentication first, **always**.*

- `Use` — adds middleware that calls the next.
- `Run` — terminal middleware; doesn't call next.
- `Map` — branches the pipeline based on path: `app.Map("/admin", admin => { /* sub-pipeline */ });`

In one project I wrote a custom middleware that logs request method, path, status code, and elapsed time, plus a correlation ID per request. With Serilog enrichers it gave us perfect distributed tracing across microservices for almost no effort.

- Request/response logging with timing.
- Tenant resolution from subdomain.
- API key validation (before built-in auth).
- Error normalization to a single JSON shape.

```
// Custom middleware — request timing & logging
public class RequestLoggingMiddleware
{
    private readonly RequestDelegate _next;
    private readonly ILogger<RequestLoggingMiddleware> _logger;

    public RequestLoggingMiddleware(RequestDelegate next, ILogger<RequestLoggingMiddleware> logger)
    {
        _next = next;
        _logger = logger;
    }

    public async Task Invoke(HttpContext ctx)
    {
        var sw = Stopwatch.StartNew();
        try
        {
            await _next(ctx);
        }
        finally
        {
            sw.Stop();
            _logger.LogInformation("{Method} {Path} responded {Status} in {Ms} ms",
                ctx.Request.Method, ctx.Request.Path, ctx.Response.StatusCode, sw.ElapsedMilliseconds);
        }
    }
}

// Register
app.UseMiddleware<RequestLoggingMiddleware>();
```

- *Where should custom exception middleware sit in the pipeline?* — At the **very top**, so it can catch errors thrown by anything below. If you place it later, errors thrown by earlier middleware bypass it.
- *Can middleware be made conditional?* — Yes — `app.UseWhen(ctx => ctx.Request.Path.StartsWithSegments("/api"), api => api.UseMiddleware<ApiAuthMiddleware>());`. Useful for branching by path or environment.

Q3: What are the most important features added in .NET 6, 7, and 8 that you've used?

This is a question that quickly separates candidates who keep up from those who don't. Here's what I'd actually mention:

— biggest developer-experience improvements in years:

- **Minimal APIs** — `app.MapGet("/", () => "Hello")`. Great for small services and prototypes.
- **Unified** `Program.cs` — no more `Startup.cs`. One file, fewer concepts.
- **Hot reload** — edit code while the app runs. Massive for productivity.
- **Implicit usings + global usings** — less ceremony.
- `DateOnly` / `TimeOnly` — finally, dates without time.

— perf and polish:

- **Rate Limiting middleware** built in (`AddRateLimiter`) — no more rolling your own.
- **Output caching middleware** — server-side response caching.
- `[FromServices]` **inferred** — no attribute needed for DI in minimal APIs.
- **Native AOT preview** for console apps.

— production-grade modern stack:

- **Native AOT for Web APIs** — fast cold start (great for serverless), smaller images, no JIT.
- **Keyed DI services** — register multiple implementations of the same interface and resolve by key. Huge for things like multiple payment providers.
- `IExceptionHandler` — clean, typed global exception handling registered via DI.
- `TimeProvider` — abstraction over `DateTime.UtcNow` so you can mock time in tests.
- `HybridCache` **(preview)** — combines in-memory + distributed cache with proper stampede protection.
- **Performance** — across the board, often 10–20% faster than .NET 7.

- **Minimal APIs** for small internal services.
- **Keyed DI** for "pick the right strategy" patterns.
- **Rate Limiter** instead of custom token-bucket code.
- `IExceptionHandler` — replaces my old custom exception middleware in new projects.

- Jumping to Native AOT for everything. It has restrictions: no runtime reflection, limited DI, slower build. Use it where cold start matters (serverless, CLI tools).
- Treating Minimal APIs as "the new way to write all APIs." For larger APIs, controllers are still cleaner — they give you filters, routing conventions, and group-level config.

```
// .NET 8 — Keyed DI
builder.Services.AddKeyedScoped<IPaymentProvider, StripeProvider>("stripe");
builder.Services.AddKeyedScoped<IPaymentProvider, RazorpayProvider>("razorpay");

public class CheckoutService(
    [FromKeyedServices("stripe")] IPaymentProvider stripe,
    [FromKeyedServices("razorpay")] IPaymentProvider razorpay)
{
    // pick provider based on user country / config
}

// .NET 8 — IExceptionHandler
public class GlobalExceptionHandler : IExceptionHandler
{
    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken ct)
    {
        ctx.Response.StatusCode = StatusCodes.Status500InternalServerError;
        await ctx.Response.WriteAsJsonAsync(new { traceId = ctx.TraceIdentifier, message = "Server error" }, ct);
        return true;
    }
}
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
app.UseExceptionHandler();
```

- *When would you choose Minimal APIs over Controllers?* — Small services with few endpoints, microservices, prototypes, and serverless functions. For larger apps with auth, filters, and conventions, controllers are cleaner.
- *What's the trade-off of Native AOT?* — Fast cold start and tiny images, but **no runtime reflection or dynamic code generation**, fewer libraries supported, and slower build times. Great for CLI/serverless, often overkill for typical Web APIs.

TOPIC 6: MVC VS WEB API

Q1: What is the difference between MVC and Web API in ASP.NET Core?

A: In ASP.NET Core, MVC and Web API live in the **same framework** — they share controllers, routing, model binding, and DI. The difference is mainly **what they return** and **how they're configured**.

Feature	MVC	Web API
Returns	HTML views (Razor)	JSON / XML data
Used for	Server-rendered websites	Backend services for SPA / mobile / 3rd-party
Base class	`Controller` (has view helpers)	`ControllerBase` (no view support)
Attribute	Usually plain	`[ApiController]`
Routing	Convention or attribute	Almost always attribute
Model state errors	Returned to view	Auto `400 BadRequest` (with `[ApiController]`)

- **Web API** — almost everything I build. SPA frontend (Angular/React), mobile apps, microservices, public APIs. JSON in, JSON out.
- **MVC** — admin panels where SEO/server rendering matters, internal tools, simple CRUD where I don't want a separate frontend.
- **Razor Pages** — even simpler than MVC for page-based scenarios. Underrated.

- Auto returns `400 BadRequest` when model validation fails.
- Requires attribute routing (no convention-based).
- Infers binding sources (`[FromBody]` for complex types, `[FromRoute]` for route params).
- Adds problem details (RFC 7807) to error responses.

Real-world example: A typical project of mine has an **admin panel in MVC** (server-rendered, simple, secure), a **Web API for the mobile/SPA**, and they share the **same `Application` and `Domain` projects**. Same business logic, two delivery mechanisms. That's the modern .NET way.

- Adding `[ApiController]` to MVC controllers that return views — breaks model binding and error handling.
- Returning HTML from a Web API. APIs return data, period.
- Mixing routing styles (some convention, some attribute) — inconsistent and confusing.

```
// MVC controller — returns a view
public class HomeController : Controller
{
    public IActionResult Index() => View(); // renders Index.cshtml
}

// Web API controller — returns data
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet]
    public IActionResult Get() => Ok(new[] { "Phone", "Laptop" });
}
```

- *Can a Web API controller return XML instead of JSON?* — Yes — add the XML formatter (`AddXmlSerializerFormatters()`) and the response format is decided by the client's `Accept` header.
- *What's the difference between `Controller` and `ControllerBase`?* — `Controller` inherits from `ControllerBase` and adds view-related helpers (`View()`, `PartialView()`, `ViewBag`). For Web APIs you don't need those, so `ControllerBase` is leaner.

TOPIC 7: DEPENDENCY INJECTION

Q1: What is Dependency Injection and why is it the foundation of modern .NET? (IMPORTANT)

A: Dependency Injection (DI) is a design pattern where **a class doesn't create its own dependencies** — they're handed to it from outside (usually through the constructor). The class only knows the **interface** it depends on, not the concrete implementation.

1. **Testability** — I can pass a mock or fake into a service in unit tests, without touching real DBs or APIs.
2. **Flexibility** — swap `EmailSender` for `MockEmailSender` in dev, `SendGridSender` in prod, without changing service code.
3. **Loose coupling** — `OrderService` depends on `IPaymentGateway`, not `StripeGateway`. The class doesn't care which one runs.
4. **Centralized wiring** — `Program.cs` is the **single place** where all dependencies are configured.
5. **Lifetime management** — the container handles creating, disposing, and sharing instances correctly.

ASP.NET Core has DI — every controller, every service, every minimal API endpoint gets dependencies injected automatically. You don't need a third-party container for 95% of apps.

in a typical service of mine, the constructor injects:

- `IUserRepository` (data),
- `IEmailService` (notifications),
- `ILogger<T>` (logging),
- `IOptions<JwtSettings>` (config).

In tests, all four are mocked. The actual service code doesn't change. That's the whole point.

- Calling `new` to create a dependency manually inside a class. Defeats DI entirely.
- Using **service locator** (resolving from `IServiceProvider` inside methods) — anti-pattern. Use constructor injection.
- Injecting `IServiceProvider` directly — same problem; hides true dependencies.

- Constructor with **too many parameters** (>5–6). That's a sign the class is doing too much (SRP violation), not a DI problem.

Modern bonus (.NET 8): **primary constructors** make DI even cleaner.

```
public interface IEmailService
{
    Task SendAsync(string to, string subject, string body);
}

public class SmtpEmailService : IEmailService
{
    public Task SendAsync(string to, string subject, string body)
    {
        // SMTP logic
        return Task.CompletedTask;
    }
}

// Classic constructor DI
public class UserService
{
    private readonly IEmailService _email;
    private readonly ILogger<UserService> _logger;

    public UserService(IEmailService email, ILogger<UserService> logger)
    {
        _email = email;
        _logger = logger;
    }

    public async Task RegisterAsync(string email)
    {
        _logger.LogInformation("Registering {Email}", email);
        await _email.SendAsync(email, "Welcome", "Hello!");
    }
}

// .NET 8 primary constructor (cleaner)
public class UserService2(IEmailService email, ILogger<UserService2> logger)
{
    public Task RegisterAsync(string e) => email.SendAsync(e, "Welcome", "Hi!");
}

// Program.cs
builder.Services.AddScoped<IEmailService, SmtpEmailService>();
builder.Services.AddScoped<UserService>();
```

- *What is Inversion of Control (IoC), and how does it relate to DI?* — IoC is the **principle**: the framework controls flow, your code plugs in. DI is **one way** to implement IoC (others include service locator, factories, etc.).
- *Can you use property injection or method injection in ASP.NET Core?* — The built-in container only supports **constructor injection**. Other styles need third-party containers. Stick with constructor injection — it makes dependencies explicit.

Q2: Singleton vs Scoped vs Transient — and what's a Captive Dependency? (IMPORTANT)

A: These are the three **service lifetimes** in ASP.NET Core's built-in DI container. Picking the wrong one is one of the most common bugs in real .NET apps.

Lifetime	One instance per	Use for	Examples
Singleton	Whole app	Stateless, thread-safe, expensive-to-create services	Caches, configuration, `IHttpClientFactory`
Scoped	One HTTP request	Services tied to request lifecycle	EF Core `DbContext`, business services
Transient	Every resolution	Lightweight, stateless	Mappers, validators, simple helpers

- Default to **Scoped** for business services and data access. It's safe and predictable.
- Use **Singleton** only for **truly shared, thread-safe** state (cache, config, logger factory).
- Use **Transient** for cheap, stateless utilities.

If you inject a **Scoped or Transient** service into a **Singleton**, the shorter-lived service gets **captured** inside the Singleton and lives **for the entire app lifetime** — completely defeating the lifetime you registered. Even worse, EF Core's `DbContext` is **not thread-safe**, so you'll get random `InvalidOperationException`s in production.

I've debugged this exact bug: someone registered a "cache helper" as Singleton, and it injected `AppDbContext`. Tests passed. Production crashed under load with random EF errors that were nearly impossible to reproduce. Took two days.

> *A service can only depend on other services with . Singleton → Singleton ✅. Scoped → Scoped or Singleton ✅. Singleton → Scoped ❌.*

- Not thread-safe.
- Holds a change tracker that's request-specific.
- Tied to a single Unit of Work per request.

- Registering `HttpClient` as Singleton and reusing it everywhere — you'll hit DNS issues. Use `IHttpClientFactory` instead.
- Registering everything as Transient "to be safe" — wastes allocations and breaks request-scoped sharing.
- Forgetting to dispose Transient services that implement `IDisposable` when resolved manually from `IServiceProvider`.

```
// Program.cs
builder.Services.AddSingleton<ICacheService, InMemoryCacheService>();
builder.Services.AddScoped<AppDbContext>();
builder.Services.AddScoped<IUserRepository, UserRepository>();
builder.Services.AddTransient<IEmailFormatter, EmailFormatter>();

// ❌ Captive dependency — DON'T DO THIS
public class CacheService : ICacheService
{
    private readonly AppDbContext _db; // Scoped injected into Singleton — CAPTURED
    public CacheService(AppDbContext db) => _db = db; // 💀
}

// ✅ Correct — inject IServiceScopeFactory and create a scope inside
public class BackgroundCleaner : ICacheService
{
    private readonly IServiceScopeFactory _scopeFactory;
    public BackgroundCleaner(IServiceScopeFactory factory) => _scopeFactory = factory;

    public async Task RunAsync()
    {
        using var scope = _scopeFactory.CreateScope();
        var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
        await db.SaveChangesAsync();
    }
}
```

- *Why is* `IHttpClientFactory` *recommended over* `new HttpClient()` *or singleton* `HttpClient`*?* — `HttpClient` is meant to be reused, but holds onto sockets. Long-lived singletons cause **stale DNS** issues. `IHttpClientFactory` pools handlers and recycles them safely.
- *How do you safely use a Scoped service from a background task?* — Create a manual scope using `IServiceScopeFactory.CreateScope()`, resolve the Scoped service inside, and dispose the scope when done.

TOPIC 8: ENTITY FRAMEWORK CORE

Q1: What is EF Core and how does Change Tracking actually work? (IMPORTANT)

A: EF Core is Microsoft's modern, lightweight, cross-platform **ORM**. It maps C# classes to database tables and lets you query with LINQ instead of writing SQL by hand. But the part that interviewers really care about is **how it tracks changes**.

When you query an entity from `DbContext`, EF Core takes a **snapshot** of its property values and stores it inside the **Change Tracker**. As you modify properties on the entity, EF compares the current values to the snapshot. When you call `SaveChanges()`, it generates the necessary `INSERT` / `UPDATE` / `DELETE` statements based on what changed. That's the magic — **you never write the SQL yourself.**

- **Added** — new entity, will be inserted.
- **Unchanged** — same as snapshot, no SQL.
- **Modified** — at least one property changed → UPDATE.
- **Deleted** — will be removed.
- **Detached** — not tracked (e.g., `AsNoTracking` queries).

- Tracking has a **memory cost** — every loaded entity is stored.
- Tracking has a **CPU cost** — every `SaveChanges` walks the entire tracked graph to detect changes.
- For **read-only queries**, tracking is **pure overhead** — use `AsNoTracking()`.

Real-world example: I had an API that loaded 10,000 rows for a report and CPU spiked at every request. After adding `.AsNoTracking()`, response time dropped from ~900 ms to ~480 ms — almost 50% faster, just because we stopped tracking entities we never planned to update.

- Using `DbContext` as a Singleton — it's **not thread-safe**, register it Scoped (one per request).
- Calling `SaveChanges` after every single change in a loop — kills perf. Batch them.
- Loading entities, mapping to DTOs, and forgetting that they're still tracked — wastes memory.
- Querying with EF and then iterating in memory — see N+1 in the next question.

```
public class AppDbContext : DbContext
{
    public DbSet<User> Users => Set<User>();
    public DbSet<Order> Orders => Set<Order>();

    protected override void OnConfiguring(DbContextOptionsBuilder o)
        => o.UseSqlServer("Server=.;Database=Shop;Trusted_Connection=True;");
}

// Tracked query — EF will detect the change automatically
var user = await _db.Users.FirstOrDefaultAsync(u => u.Id == 5);
user!.Name = "Updated";
await _db.SaveChangesAsync(); // generates UPDATE Users SET Name=... WHERE Id=5
```

- *What's the difference between EF Core and Dapper?* — **EF Core** is a full ORM with change tracking, migrations, LINQ-to-SQL translation. **Dapper** is a tiny micro-ORM — you write SQL, it maps results to objects. Dapper is faster and simpler; EF Core is more productive for CRUD-heavy work.
- *Can EF Core work with stored procedures?* — Yes, via `FromSqlRaw` / `FromSqlInterpolated` / `ExecuteSqlRaw`. Use them when LINQ can't express what you need or when you need DBA-tuned SQL.

Q2: What is AsNoTracking, the N+1 problem, and how do you avoid it? (IMPORTANT)

A: Two of the **biggest** EF Core perf killers in real apps. Senior devs are expected to know both cold.

`AsNoTracking()` tells EF Core *"I'm only reading this data — don't bother tracking it."* Skips the snapshot, saves CPU and memory. For read-heavy endpoints, this can easily give **30–50%** improvement with one line.

```
// Read-only — use AsNoTracking
var products = await _db.Products
    .AsNoTracking()
    .Where(p => p.IsActive)
    .ToListAsync();

// Need to update — DON'T use AsNoTracking (changes won't be saved)
var user = await _db.Users.FirstAsync(u => u.Id == id);
user.Name = "Updated";
await _db.SaveChangesAsync();
```

The N+1 problem: when you load a list of N entities and **lazily load** a related entity for each one, EF makes **1 query for the list + N queries for the related data**. For 1000 orders, that's **1001 SQL roundtrips**.

I've seen N+1 take an endpoint from 80 ms to in production. The fix is almost always:

1. **Eager loading with** `Include` — single SQL JOIN.
2. **Projection with** `Select` — pull only the columns you need into a DTO.

because it avoids loading entire entity graphs and the overhead of materializing them as tracked entities.

```
// ❌ N+1 — lazy loading triggers a query per order
foreach (var order in await _db.Orders.ToListAsync())
{
    Console.WriteLine(order.Customer.Name); // SQL for each order!
}

// ✅ Fix 1 — eager load with Include (one JOIN query)
var orders = await _db.Orders
    .AsNoTracking()
    .Include(o => o.Customer)
    .ToListAsync();

// ✅ Fix 2 — projection (best for read APIs)
var dtos = await _db.Orders
    .AsNoTracking()
    .Select(o => new OrderDto(o.Id, o.Customer.Name, o.Total))
    .ToListAsync();
```

- **Calling** `.ToList()` **early** then filtering in memory — make sure filtering happens in SQL.
- `.Count()` **on huge tables** — generates `SELECT COUNT(*)`; fine, but know it scans.
- **Excessive** `.Include`**s** — **cartesian explosion**. EF 5+ has `AsSplitQuery()` to split into multiple queries.

- **Long-running** `DbContext` — request-scoped only.
- **Bulk operations** — EF tracks each entity, so saving 10K rows is slow. Use **EF Core 7+** `ExecuteUpdate` / `ExecuteDelete` or libraries like `EFCore.BulkExtensions`.
- Using `AsNoTracking()` and then trying to update — silent failure.
- Adding `.Include()` to a write query — slows save and adds load.
- Over-projecting (selecting unnecessary columns) — defeats the perf benefit.
- *What is* `AsSplitQuery` *and when would you use it?* — When using multiple `.Include`s, EF generates one big JOIN that can return **massive duplicate data** (cartesian explosion). `AsSplitQuery()` runs each `Include` as a separate query, dramatically faster for many child collections.
- *How do* `ExecuteUpdate` *and* `ExecuteDelete` *(EF Core 7+) help?* — They generate a **single SQL statement** that bypasses change tracking. Perfect for bulk updates: `await _db.Orders.Where(o => o.Old).ExecuteDeleteAsync();` — one query, no entity loading.

Q3: How do migrations work and what are common EF Core mistakes you've seen?

A: Migrations are EF Core's way of **versioning your database schema** alongside your C# code. Every model change generates a migration file (a class that describes the schema diff), and applying the migration runs SQL to bring the DB up to date.

1. Modify the model (add a property, new entity, change a relationship).
2. `dotnet ef migrations add <DescriptiveName>` — generates a migration file.
3. **Review the generated SQL** — `dotnet ef migrations script` — never trust it blindly.
4. `dotnet ef database update` — applies migrations to the local DB.
5. Commit the migration files to Git — they're part of the codebase.
6. Apply automatically in CI/CD on deploy.

- **Always review migrations before applying** — EF can drop columns / rename in destructive ways.
- **Never edit an applied migration** — create a new one.
- **Rename migrations** to be **descriptive** (`AddOrderStatusEnum`, not `Migration1`).
- **Don't use** `EnsureCreated()` **in production** — it bypasses migrations.
- **For prod**, prefer generating SQL scripts (`dotnet ef migrations script`) and reviewing with the DBA.

1. **No connection pooling** — every request opens a new connection. Use `AddDbContextPool` for high-throughput APIs.
2. **Lazy loading enabled by default** — looks innocent, causes N+1 everywhere. I disable it.
3. **Async/sync mixing** — `db.Users.ToList()` instead of `ToListAsync()` blocks threads.
4. **Generic repository over EF** — adds a layer with no value. EF's `DbSet<T>` is already a repository.
5. `SaveChanges` **per row in a loop** — each call is a roundtrip + transaction.
6. **No indexes on FK columns** — EF doesn't always create them; check the generated migration.
7. **String comparisons that don't translate to SQL** — `.Where(u => u.Name.Equals(name, StringComparison.OrdinalIgnoreCase))` blows up. Use `EF.Functions.Like` or DB collation.
8. **Disposing** `DbContext` **while async work is in flight** — race conditions.

Real-world example: in a project I joined, the report API took 12 seconds. Root cause: `GetReports()` called 8 endpoints, each loaded the entire `Order` entity with all 30 columns, no `AsNoTracking`, no projection, and lazy loading triggered N+1 on `Customer`. After fixing — projection + `AsNoTracking` + `AsSplitQuery` for the one needed `Include` — same endpoint dropped to **~700 ms**. **17× faster** with no infrastructure change.

```
# Standard migration workflow
dotnet ef migrations add InitialCreate
dotnet ef database update

# Generate SQL script (review before applying to prod)
dotnet ef migrations script -o migration.sql

# Roll back to a specific migration
dotnet ef database update PreviousMigrationName
```

- *What's the difference between `AddDbContext` and `AddDbContextPool`?* — `AddDbContext` creates a new `DbContext` per request. `AddDbContextPool` reuses pooled instances — faster for high-traffic APIs, but you must avoid storing per-request state in the context.
- *How do you handle DB schema changes on a multi-instance production deployment?* — Apply migrations during deployment **before** new app instances start, write **backwards-compatible** migrations (add new columns, then deploy code, then remove old in a later release). This is called **expand/contract** migration.

TOPIC 9: LINQ

Q1: What is the difference between IEnumerable and IQueryable? (IMPORTANT)

This is the LINQ question that separates juniors from people who actually understand how the data layer works.

Feature	IEnumerable<T>	IQueryable<T>
Where it executes	In **memory** (LINQ to Objects)	At the **data source** (translated to SQL by EF Core)
Namespace	`System.Collections.Generic`	`System.Linq`
Builds	Iterator (delegates)	**Expression tree**
Use when	Working with collections you already have	Querying a database
Risk	Can pull entire dataset into memory if used too late	Some LINQ can't be translated to SQL → runtime error

Mental model: `IQueryable` is a **plan** for a query — EF Core can read the expression tree and translate it to SQL. `IEnumerable` is a stream — once you hand it to a method that returns `IEnumerable`, the data **must already be in memory**.

The trap: if you call any method that returns `IEnumerable` on a queryable, **everything after it runs in memory**.

a junior on my team wrote:

```
var users = _db.Users.AsEnumerable().Where(u ⇒ u.IsActive).ToList();
```

That `.AsEnumerable()` pulled **all 250,000 users from the DB into memory**, then filtered in C#. The endpoint timed out under load. Removing `.AsEnumerable()` made it `WHERE IsActive = 1` in SQL — milliseconds.

> *Stay in `IQueryable` as **long as possible**, materialize (`ToList`, `ToArray`, `FirstAsync`, etc.) only at the very end.*

- Mixing `IEnumerable` filters into the middle of EF queries — silently breaks SQL translation.

- Returning `IEnumerable<T>` from repository methods that should return `IQueryable<T>` (or DTOs) — limits caller's ability to filter at DB level.
- Calling `.ToList()` "to be safe" before the final filter — defeats the database.

```
// ✅ IQueryable — runs as SQL
var activeUsers = await _db.Users
    .Where(u ⇒ u.IsActive)          // SQL WHERE
    .OrderBy(u ⇒ u.Name)            // SQL ORDER BY
    .Take(20)                       // SQL TOP 20
    .ToListAsync();

// ❌ Pulls everything into memory before filtering
var bad = _db.Users.AsEnumerable()
    .Where(u ⇒ u.IsActive)
    .ToList();
```

- *Can a method that takes `IEnumerable<T>` accept an `IQueryable<T>`?* — Yes (interface inheritance), but **the moment it does, you're stuck in memory** for the rest of the chain. Be deliberate about return types in repositories.
- *Why is exposing `IQueryable` from a repository sometimes considered bad?* — It leaks data-access concerns up to the caller — they can do anything, including expensive queries. The alternative is returning **specific methods or DTOs**. Trade-off between flexibility and encapsulation.

Q2: Explain Deferred vs Immediate execution in LINQ.

A: Most LINQ operators are **deferred** — they don't actually run when you call them. They build up a **query definition** that runs only when you **iterate or materialize** the result. This is the source of many subtle bugs.

Type	Operators	When does it execute?
Deferred	`Where`, `Select`, `OrderBy`, `Take`, `Skip`, `Distinct`, `GroupBy`	When you iterate (`foreach`, `ToList`, etc.)
Immediate	`ToList`, `ToArray`, `ToDictionary`, `Count`, `First`, `Single`, `Any`, `Sum`	Right when called

1. The **source can change** between defining the query and iterating it.
2. The query can **run multiple times** if you iterate it more than once → multiple DB hits in EF.
3. Captured variables can change → "closure capture" bugs.

Real-world example: I had a bug where I defined `var products = _db.Products.Where(p ⇒ p.IsActive)` (no `.ToListAsync`) and used it in two different `foreach` loops. Each loop **hit the database again** — same query, twice. Materializing once with `.ToListAsync()` cut DB load in half.

Another classic: the closure capture bug.

```
var queries = new List<IEnumerable<int>>();
for (int i = 0; i < 3; i++)
    queries.Add(nums.Where(n ⇒ n > i)); // captures `i`, not its value!
foreach (var q in queries) Console.WriteLine(q.Count()); // all use i=3
```

- Iterating an `IQueryable` multiple times → multiple SQL queries.
- Caching an `IQueryable` — every consumer hits the DB. Cache `List<T>` instead.

- Modifying the source collection between query definition and iteration.

```
var nums = new List<int> { 1, 2, 3 };

// Deferred — query is just a plan
var query = nums.Where(n => n > 1);
nums.Add(5);
foreach (var n in query) Console.WriteLine(n); // 2, 3, 5 — sees the new item!

// Immediate — runs now, snapshot
var snapshot = nums.Where(n => n > 1).ToList();
```

- `.Any()` is faster than `.Count() > 0` for `IEnumerable` because it stops at the first match.
- `.Count` (the **property** on `List<T>`) is O(1); `.Count()` (LINQ method on `IEnumerable`) can be O(N).
- *Why might calling `.Count()` twice in a row on an `IEnumerable` be problematic?* — It iterates **twice** — and if the source is a query, that's two DB calls or two file reads. Always materialize first if you need to use the result more than once.
- *How does `yield return` relate to deferred execution?* — `yield return` creates a state machine that produces values on demand — it's the building block of deferred LINQ. Method results are computed only when the consumer iterates.

TOPIC 10: ASYNC / AWAIT

Q1: Explain async/await and how it actually works under the hood. (IMPORTANT)

A: `async` and `await` are C# keywords that let us write **non-blocking I/O code** that reads like synchronous code. They're not about making things faster per call — they're about making the app **scale** by not wasting threads while waiting.

The big idea: when your code is waiting on I/O (DB, HTTP, file, etc.), the thread doesn't sit idle. It's released back to the thread pool to handle **other requests**, and your method **resumes** automatically when the I/O completes.

Under the hood: the compiler rewrites your `async` method into a **state machine**. Every `await` becomes a state transition. When the awaited operation finishes, a continuation runs your remaining code — possibly on a different thread.

Why it matters in Web APIs: a sync method that blocks on a DB call holds onto a thread for the full DB roundtrip. With async, that thread serves another request while waiting. On a 100-request/sec API with 50 ms DB calls, this can be the difference between **handling all traffic** and **the thread pool exhausting and timing out**.

I migrated a sync API to async. Throughput on the same hardware went from ~120 RPS to ~750 RPS. The work per request didn't change — we just stopped holding threads hostage during I/O.

1. **"Async all the way"** — once you go async, never block with `.Result` or `.Wait()`.
2. **Always end I/O method names with** `Async` — `GetUsersAsync`, `SaveOrderAsync`. Helps callers spot async methods.
3. **Use** `Task<T>` **or** `ValueTask<T>` **as return type** — never `async void` (except event handlers).
4. **Pass** `CancellationToken` through the call chain. This is huge for scaling — cancelled requests stop wasting work.
5. **Use** `await` **instead of returning the task directly** when you need correct exception/stack trace handling. Returning the task directly is faster but loses some debugging context.

- `.Result` / `.Wait()` on async methods → can deadlock (see next question).
- `async void` everywhere → exceptions become unobservable; can crash the app.
- Wrapping CPU-bound work in `Task.Run` inside ASP.NET — wastes threads, doesn't scale.
- Not flowing `CancellationToken` — your code keeps running even after the client disconnected.

```
// ✅ Modern async controller
[HttpGet("{id}")]
public async Task<IActionResult> GetUser(int id, CancellationToken ct)
{
    var user = await _db.Users
        .AsNoTracking()
        .FirstOrDefaultAsync(u => u.Id == id, ct);

    return user is null ? NotFound() : Ok(user);
}

// ❌ Common anti-patterns
public IActionResult Bad(int id)
{
    var user = _db.Users.FirstAsync(u => u.Id == id).Result; // deadlock risk
    return Ok(user);
}
```

- *What's the difference between* `Task` *and* `ValueTask`*?* — `Task` is a class (heap allocation); `ValueTask` is a struct (no allocation when result is already available). Use `ValueTask<T>` for hot paths that often complete synchronously (e.g., cache hits). Use `Task` everywhere else.
- *What does* `Task.WhenAll` *do and when do you use it?* — Runs multiple tasks **in parallel** and awaits all. Great for fan-out scenarios — e.g., calling 5 independent APIs at once instead of sequentially. Cuts total wait time to the slowest one.

Q2: What is the classic async deadlock and what does ConfigureAwait(false) do?

This is a famous question because it catches almost everyone, and senior devs are expected to explain it crisply.

1. Your code calls an async method and **blocks on** `.Result` in a context that has a **synchronization context** (UI threads, classic ASP.NET).
2. The async method tries to **resume on the original context** after `await`.
3. But that context is **already blocked** waiting for `.Result`.
4. → No one can move. **Deadlock.**

The fix isn't complicated — . "Async all the way."

Where this matters today: ASP.NET Core does **not have a synchronization context**, so this specific deadlock is much rarer there. But you'll still see it in:

- WPF, WinForms, MAUI / Xamarin (UI threads).
- Classic ASP.NET (.NET Framework).
- Library code that doesn't know what host it's running in.

`ConfigureAwait(false)`**:** tells the awaited task *"I don't need to resume on the captured context."* In library code, this prevents the deadlock and is also slightly faster (no context switch). In **ASP.NET Core app code**, it's mostly **noise** — there's no context to capture anyway.

> *Use* `ConfigureAwait(false)` *in* ***library code*** *that might be called from any host. Skip it in* ***ASP.NET Core app code*** *— it's not needed.*

- `async void` — fire-and-forget that swallows exceptions. Only legal use is event handlers.
- `Task.Run` **to "make sync code async"** — pushes work to a thread pool thread. Doesn't add scalability; might even hurt it inside ASP.NET. Use it only for **CPU-bound** work, not I/O.

- **Forgetting to `await`** — `Task something = SomeAsync();` without await is a fire-and-forget. Compiler warns; never ignore the warning.
- `Task.Result` **in property getters** — terrible for testing and async-unfriendly. Make the property a method.

Real-world example: A junior added `var data = GetDataAsync().Result;` in a WPF button click handler. The UI froze every time. The fix took 30 seconds — make the click handler `async` and `await`. Their tests didn't catch it because the test runner didn't have a UI sync context.

```
// ❌ Deadlock-prone — blocks on async with sync context
public string GetTitle()
    => GetTitleAsync().Result;

// ✅ Async all the way
public async Task<string> GetTitleAsync()
{
    var html = await _http.GetStringAsync("https://example.com");
    return ExtractTitle(html);
}

// ✅ Library code — ConfigureAwait(false)
public async Task<string> GetTitleLibAsync()
{
    var html = await _http.GetStringAsync("https://example.com").ConfigureAwait(false);
    return ExtractTitle(html);
}

// ✅ Parallel I/O with WhenAll
var tasks = new[] { GetUserAsync(1), GetUserAsync(2), GetUserAsync(3) };
var users = await Task.WhenAll(tasks);
```

- *When is it OK to use `Task.Run`?* — For **CPU-bound** work that you want to offload from the request thread. For I/O, never — async I/O doesn't need a thread, so `Task.Run` actively hurts scalability.
- *How does `await` propagate exceptions?* — Awaiting a faulted task **rethrows the exception** at the await point. Multiple exceptions (e.g., from `WhenAll`) are wrapped in `AggregateException`, but `await` unwraps the first one — use `.Exception` to access all.

TOPIC 11: EXCEPTION HANDLING & LOGGING

Q1: How should you handle exceptions in real .NET applications? (IMPORTANT)

A: Exception handling is one of those areas where the difference between juniors and seniors is huge. Anyone can write `try { ... } catch { }`. The senior question is: **what's the strategy?**

1. **Catch specific exceptions, not `Exception`.** Catching the base type hides bugs.
2. **Don't swallow exceptions silently.** Empty `catch {}` is the most evil pattern in .NET.
3. **Use `throw;` to re-throw, never `throw ex;`.** The latter resets the stack trace.
4. **Don't use exceptions for control flow.** They're slow (~100× slower than `if`) and obscure intent.
5. **Always log with context** — message, exception, request ID, user ID, the inputs that caused it.
6. **Prefer `using` over `try/finally`** for `IDisposable`. Cleaner and impossible to forget.
7. **Use `when` filters** to inspect exceptions without catching: `catch (SqlException ex) when (ex.Number == 2627)`.
8. **Rely on global handling for cross-cutting concerns** (next question), keep local `try/catch` for **specific recovery**.

- You can **recover** (retry, fallback, default value).
- You need to **wrap** the exception in a domain-specific one.
- You need to **add context** to the log.

- You can't do anything useful — let it bubble up to the global handler.
- You'd just `Console.WriteLine` it — that's worse than crashing.

- `catch (Exception) { }` — this is how production silently breaks.
- `try/catch` around every method "just in case" — clutters code, hides errors.
- Throwing `Exception`, not specific types — callers can't catch by type.
- Forgetting that **async exceptions** are stored on the `Task` — you only see them when you `await`.

Custom exceptions: great for **domain errors** (`InsufficientStockException`, `InvalidCouponException`). They let API layers map them to specific HTTP status codes cleanly.

```
// Specific catches with context
try
{
    var json = await File.ReadAllTextAsync("config.json");
    var data = JsonSerializer.Deserialize<Config>(json);
    return data!;
}
catch (FileNotFoundException ex)
{
    _logger.LogError(ex, "Config file missing at {Path}", "config.json");
    throw new ConfigurationException("Config file not found", ex);
}
catch (JsonException ex)
{
    _logger.LogError(ex, "Config JSON is invalid");
    throw new ConfigurationException("Config malformed", ex);
}

// when filter — catch only specific SQL errors
catch (SqlException ex) when (ex.Number == 2627) // unique constraint
{
    throw new DuplicateEntryException("Item already exists", ex);
}
```

- *Why is `throw;` better than `throw ex;`?* — `throw;` preserves the **original stack trace** and exception details. `throw ex;` resets the stack trace to the current line, making it almost impossible to find the original failure.
- *What's the performance cost of throwing exceptions?* — Significant — exceptions involve stack walking and capture. They're roughly 100–1000× slower than a simple `if` check. Never use them for normal flow control.

Q2: How do you do global exception handling and structured logging in ASP.NET Core?

A: Wrapping every controller in `try/catch` is repetitive, error-prone, and ugly. ASP.NET Core gives us several **centralized** ways to handle exceptions, log them properly, and return a clean, consistent error response.

1. `UseExceptionHandler` **middleware** — built-in. Routes errors to a handler endpoint or callback.
2. **Custom exception middleware** — your own `IMiddleware` class. Full control.
3. `IExceptionHandler` **(.NET 8+)** — clean, DI-registered handlers. **My current favorite.**

```
{
  "type": "https://example.com/problems/validation",
  "title": "Validation failed",
  "status": 400,
  "traceId": "00-abc...",
  "errors": { "Name": ["Required"] }
}
```

This is the standard the modern .NET ecosystem uses. Frontends, mobile, and 3rd-party consumers all expect it. Don't invent your own format.

- `ILogger<T>` is the standard interface. **Always inject it**, never use `Console.WriteLine`.
- **Use structured logging** — log properties, not strings: `_logger.LogInformation("User {UserId} placed order {OrderId}", userId, orderId);`. Searchable in Seq/ELK/Datadog.
- **Use Serilog** for production — JSON output, file/Elasticsearch sinks, enrichers.
- **Correlation ID per request** — attach to every log so you can trace a single request across services.
- **Log levels matter:** Trace → Debug → Information → Warning → Error → Critical. **Don't log everything as Error.**
- **Never log PII** (passwords, tokens, full credit cards) — compliance nightmare.
- **Don't log inside hot loops.**

Real-world example: in production we log every error with `traceId`, `userId`, `endpoint`, and the exception. When a user reports a bug, we ask for the `traceId` shown in the error response, search Seq, and see the full request flow within seconds. That setup probably saves us 5+ hours per week.

- Returning **raw stack traces** to clients in production. Leaks paths, library versions, sometimes secrets.
- String-concat logging (`"User " + id + " did X"`) — kills structured search.
- Logging at the wrong level — `LogError` for expected validation failures fills your alerting noise.
- No correlation ID — debugging distributed systems becomes archaeology.

```
// .NET 8 — IExceptionHandler (clean modern way)
public class GlobalExceptionHandler : IExceptionHandler
{
    private readonly ILogger<GlobalExceptionHandler> _logger;
    public GlobalExceptionHandler(ILogger<GlobalExceptionHandler> logger) => _logger = logger;

    public async ValueTask<bool> TryHandleAsync(HttpContext ctx, Exception ex, CancellationToken ct)
    {
        _logger.LogError(ex, "Unhandled exception for {Path}", ctx.Request.Path);

        ctx.Response.StatusCode = ex switch
        {
            ValidationException => StatusCodes.Status400BadRequest,
            NotFoundException   => StatusCodes.Status404NotFound,
            UnauthorizedAccessException => StatusCodes.Status401Unauthorized,
            _                   => StatusCodes.Status500InternalServerError
        };

        await ctx.Response.WriteAsJsonAsync(new
        {
            traceId = ctx.TraceIdentifier,
            title   = "An error occurred",
            status  = ctx.Response.StatusCode,
            message = ex is DomainException ? ex.Message : "Please try again later"
        }, ct);

        return true;
    }
}

// Program.cs
builder.Services.AddExceptionHandler<GlobalExceptionHandler>();
builder.Services.AddProblemDetails();
app.UseExceptionHandler();
```

- *What is structured logging and why does it matter?* — Logging properties (`{UserId}`) instead of pre-formatted strings, so log aggregators can index, filter, and search them. Without structured logging, you're grepping text — useless at scale.
- *How would you implement request correlation across microservices?* — Pass a `traceparent` header (W3C Trace Context) on every outbound HTTP call, and enrich every log with it. ASP.NET Core's `Activity` API and OpenTelemetry give you this for free if configured.

TOPIC 12: WEB API (ROUTING, FILTERS, STATUS CODES, VERSIONING)

Q1: How does Routing and Model Binding work in ASP.NET Core Web API?

A: Routing maps an **HTTP request URL** to a **controller action**. Model binding takes **request data** (route, query, body, headers, form) and **maps it into method parameters**.

Routing: in Web API, I always use **attribute routing** — explicit, visible, and flexible.

```
[ApiController]
[Route("api/[controller]")]
public class ProductsController : ControllerBase
{
    [HttpGet]                          // GET /api/products
    public IActionResult GetAll() => Ok();

    [HttpGet("{id:int}")]              // GET /api/products/5
    public IActionResult Get(int id) => Ok();

    [HttpGet("by-name/{name:alpha}")] // GET /api/products/by-name/phone
    public IActionResult GetByName(string name) => Ok();

    [HttpPost]                         // POST /api/products
    public IActionResult Create([FromBody] ProductDto dto)
        => CreatedAtAction(nameof(Get), new { id = 1 }, dto);
}
```

Route constraints: `{id:int}`, `{slug:alpha}`, `{date:datetime}`, `{name:minlength(3)}`. They prevent ambiguous matches and give better 404s instead of unexpected method calls.

Model binding sources (with `[ApiController]`, these are inferred automatically):

- `[FromRoute]` — from the URL path (`/api/users/{id}`).
- `[FromQuery]` — from `?key=value` query string.
- `[FromBody]` — JSON body (only one allowed per action).
- `[FromHeader]` — request headers.
- `[FromForm]` — multipart form data (uploads).
- `[FromServices]` — inject a service into a single action (alternative to constructor).

Validation: with `[ApiController]`, model state is validated automatically and returns `400 BadRequest` with details if invalid. No need for `if (!ModelState.IsValid)` boilerplate.

- Creating two endpoints with the same URL pattern — `Ambiguous match` exception at runtime. Add route constraints.
- Forgetting `[FromBody]` for complex types in older code (modern `[ApiController]` infers it).
- Using **convention-based routing** in Web API. Stick with attribute routing — it's clearer.
- Returning `IActionResult` everywhere even when you know the type — modern code uses `ActionResult<T>` for better OpenAPI/Swagger generation.

Real-world tip freshers don't know: `[ApiController]` does a lot under the hood. It's not just "a marker attribute" — it changes how validation, binding, and responses work.

```
// Modern action with proper return type
[HttpGet("{id:int}")]
public async Task<ActionResult<ProductDto>> Get(int id, CancellationToken ct)
{
    var product = await _service.GetAsync(id, ct);
    return product is null ? NotFound() : Ok(product);
}
```

- *What's the difference between attribute routing and convention-based routing?* — Attribute routing puts route info **on the action**; convention routing uses templates registered in `Program.cs`. Attribute routing is the standard for Web APIs because it's more explicit and easier to reason about.
- *Can you have route parameters with default values?* — Yes: `[HttpGet("{page:int=1}")]`. Useful for pagination defaults without separate endpoints.

Q2: What HTTP status codes should you use in a RESTful API? (IMPORTANT)

Returning the right status code is part of being a good API citizen. It's also a senior expectation — clients (frontend, mobile, integration partners) react based on these codes.

Range	Meaning	Common Codes
2xx	Success	200, 201, 202, 204
3xx	Redirection	301, 302, 304
4xx	Client error	400, 401, 403, 404, 405, 409, 422, 429
5xx	Server error	500, 502, 503, 504

- `200 OK` — Success with response body.
- `201 Created` — New resource created. Return its URL in the `Location` header.
- `202 Accepted` — Request accepted but not finished (async/queued processing).
- `204 No Content` — Success, no body (typical for `DELETE` / `PUT`).
- `400 Bad Request` — Malformed request / failed validation.
- `401 Unauthorized` — **Not authenticated** (no/invalid token).
- `403 Forbidden` — Authenticated but **not allowed**.
- `404 Not Found` — Resource doesn't exist.
- `409 Conflict` — State conflict (duplicate, version mismatch).
- `422 Unprocessable Entity` — Well-formed but business rules failed.
- `429 Too Many Requests` — Rate limit hit.
- `500 Internal Server Error` — Unexpected server crash.
- `502 Bad Gateway` / `503 Service Unavailable` / `504 Gateway Timeout` — Infra issues.

- **401** = "I don't know who you are." → Client should reauthenticate.
- **403** = "I know you, but you're not allowed." → Client shouldn't retry; they need different permissions.

- Returning `200 OK` for everything, with `success: false` in the body. Wrong. Use proper codes.
- Using `500` for validation errors. That's `400` / `422`.
- Returning `404` for "not found in this user's data" (could leak existence). Sometimes `403` is safer.
- Never returning `201` after creating something. The standard expects it.

Real-world example: I built an order creation endpoint. If the body is invalid → `400`. If the user isn't logged in → `401`. If the user is logged in but isn't the customer → `403`. If the cart is empty → `422` with a domain-specific code. If the same idempotency key was used → `409`. If everything succeeds → `201` with the new order URL. Frontend devs love this — they can react properly to every case.

```
[HttpPost]
public async Task<IActionResult> Create(OrderDto dto, CancellationToken ct)
{
    if (!ModelState.IsValid)
        return BadRequest(ModelState);                              // 400

    if (await _orders.ExistsByIdempotencyKey(dto.IdempotencyKey, ct))
        return Conflict("Duplicate request");                       // 409

    if (dto.Items.Count == 0)
        return UnprocessableEntity("Cart is empty");                // 422

    var order = await _orders.CreateAsync(dto, ct);
    return CreatedAtAction(nameof(Get), new { id = order.Id }, order); // 201
}

[HttpDelete("{id:int}")]
public async Task<IActionResult> Delete(int id, CancellationToken ct)
{
    var ok = await _orders.DeleteAsync(id, ct);
    return ok ? NoContent() : NotFound();                           // 204 / 404
}
```

- *What's the difference between 400 and 422?* — `400` = the request is **malformed** (bad JSON, missing required field). `422` = the request is well-formed but **business rules** failed (e.g., out of stock).
- *When would you return 202 Accepted?* — For async operations that take long enough to complete asynchronously — file uploads, report generation, large imports. Return a status URL so the client can poll progress.

Q3: What are Filters in ASP.NET Core and when do you use them?

A: Filters run **before or after** an action method and let you handle **cross-cutting concerns** like logging, validation, exception handling, caching, and authorization — without polluting every controller with the same code.

Filter	Runs	Common use
Authorization	First — before model binding	Permission checks
Resource	Around the entire pipeline (after auth, before model binding)	Caching, short-circuiting
Action	Before/after the action executes	Logging, request enrichment
Exception	When the action throws	Centralized error handling
Result	Before/after the response is written	Modify final response

- **Middleware** runs for every request, doesn't know about MVC concepts (controller, action, model state).
- **Filters** run only for MVC actions and **have access** to the action descriptor, parameters, model state, and result.

Rule: if you need MVC context (model state, action arguments), use a **filter**. Otherwise, use **middleware** — it's cleaner and faster.

- **Validation filter** — checks `ModelState.IsValid` and returns 400 (though `[ApiController]` does this automatically now).
- **Audit filter** — logs who called what action with what arguments.
- **Caching filter** — caches GET responses based on action + params.
- **Permission filter** — checks per-action authorization.

- Using filters for things middleware does better (e.g., global request logging). Wastes MVC overhead.
- Ignoring **filter execution order** — filter ordering can be set with the `Order` property.
- Using `IFilterFactory` when a simple `[ServiceFilter]` would do.
- Doing async work in `OnActionExecuting` when the async version (`OnActionExecutionAsync`) exists — blocks the thread.

```
// Logging filter
public class LogActionFilter : IAsyncActionFilter
{
    private readonly ILogger<LogActionFilter> _logger;
    public LogActionFilter(ILogger<LogActionFilter> logger) => _logger = logger;

    public async Task OnActionExecutionAsync(ActionExecutingContext ctx, ActionExecutionDelegate next)
    {
        var sw = Stopwatch.StartNew();
        _logger.LogInformation("⟶ {Action}", ctx.ActionDescriptor.DisplayName);

        var executed = await next();

        sw.Stop();
        _logger.LogInformation("⟵ {Action} ({Ms} ms, status {Status})",
            ctx.ActionDescriptor.DisplayName,
            sw.ElapsedMilliseconds,
            executed.HttpContext.Response.StatusCode);
    }
}

// Register globally
builder.Services.AddScoped<LogActionFilter>();
builder.Services.AddControllers(o => o.Filters.Add<LogActionFilter>());

// Or per-action
[ServiceFilter(typeof(LogActionFilter))]
[HttpGet] public IActionResult Get() => Ok();
```

- *What's the difference between `[ServiceFilter]` and `[TypeFilter]`?* — `[ServiceFilter]` resolves the filter from DI (it must be registered). `[TypeFilter]` creates an instance per use without requiring DI registration. Prefer `[ServiceFilter]` for consistency and lifetimes.
- *Can a filter short-circuit a request?* — Yes — set `context.Result` in an authorization, resource, or action filter, and the action is skipped.

Q4: How do you handle API Versioning?

A: Versioning matters the moment you have **external clients** (mobile apps, third-party integrations) that you can't update in lockstep with the API. Without versioning, every breaking change becomes an outage.

Strategy	Looks like	Pros	Cons
URL path	`/api/v1/users`	Visible, cacheable, simple	URL changes per version
Query string	`/api/users?api-version=1.0`	Easy to add	Less RESTful
Header	`api-version: 1.0`	Clean URLs	Less discoverable
Media type	`Accept: application/vnd.myapi.v1+json`	"Pure REST"	Hardest to use

My production preference: **URL path versioning** (`/api/v1/`). Pros: visible in logs, easy to route, easy for consumers to understand. The slight ugliness is worth the clarity.

The standard library: `Asp.Versioning.Mvc` (Microsoft maintained). It supports all four strategies and has a great Swagger integration.

- **Version from day one** — even if it's just `v1`. Adding versioning later is painful.
- **Bump major version only for breaking changes** — backwards-compatible additions don't need a new version.
- **Deprecate old versions explicitly** — return `Deprecation` and `Sunset` HTTP headers; document the EOL date.
- **Keep N and N-1** versions live, no more. Maintaining 5 versions in parallel is a nightmare.
- **Don't version internal APIs** — they should be deployed together with consumers; versioning is for **external** APIs.

Real-world example: I worked on a public REST API consumed by ~40 partner integrations. We versioned via URL path. When we changed the response shape of `/api/users`, we kept `/api/v1/users` running for 6 months and added `/api/v2/users` with the new shape. Partners migrated at their own pace. Zero outages.

- "We'll add versioning when we need it." You'll need it sooner than you think.
- Bumping versions for non-breaking changes (e.g., adding a new optional field). Wasteful.
- No deprecation notice — partners discover their integration is broken in production.
- Mixing versioning strategies in the same API. Pick one and stick with it.

```
// Setup (Asp.Versioning.Mvc)
builder.Services.AddApiVersioning(o =>
{
    o.DefaultApiVersion = new ApiVersion(1, 0);
    o.AssumeDefaultVersionWhenUnspecified = true;
    o.ReportApiVersions = true;            // adds `api-supported-versions` header
    o.ApiVersionReader = new UrlSegmentApiVersionReader();
}).AddMvc();

[ApiController]
[ApiVersion("1.0")]
[ApiVersion("2.0")]
[Route("api/v{version:apiVersion}/[controller]")]
public class UsersController : ControllerBase
{
    [HttpGet, MapToApiVersion("1.0")]
    public IActionResult GetV1() => Ok(new { version = "v1" });

    [HttpGet, MapToApiVersion("2.0")]
    public IActionResult GetV2() => Ok(new { version = "v2", newField = "added" });
}
```

- *How do you avoid duplicating logic across versions?* — Keep the **business logic in services** that both versions call. Only the **shape** of the response (DTO mapping) changes between versions.
- *When would you choose header versioning over URL versioning?* — When you want **clean, version-free URLs** and your consumers are sophisticated enough to set headers correctly. Less common in practice.

TOPIC 13: AUTHENTICATION & AUTHORIZATION

Q1: Explain JWT Authentication in ASP.NET Core. (IMPORTANT)

A: JWT (**JSON Web Token**) is the standard for **stateless authentication** in modern Web APIs. Instead of the server storing session state, the **client carries a signed token** with every request, and the server just **verifies the signature** to trust it.

1. User calls `POST /login` with credentials.
2. Server validates them, then issues a **JWT** signed with a secret key.
3. Client stores the JWT (memory, httpOnly cookie, or secure storage) and sends it in the `Authorization: Bearer <token>` header on subsequent requests.
4. ASP.NET Core's JWT middleware validates the signature, expiration, issuer, audience.
5. If valid, the request gets a `User` principal with claims; controllers can use `[Authorize]`.

```
HEADER.PAYLOAD.SIGNATURE
```

- **Header** — algorithm + type.
- **Payload** — **claims** (user ID, roles, expiration). **Not encrypted** — anyone can read it.
- **Signature** — HMAC of header+payload with the secret key. Server verifies this; client can't forge it.

- **Stateless** — server doesn't need to track sessions; great for horizontal scaling.
- **Cross-service** — multiple microservices can verify the same token without sharing a session store.
- **Standard** — every framework supports it.

- **Tokens are visible to anyone who has them.** Don't put sensitive data in claims.
- **Tokens can't be revoked easily.** If a user logs out or you ban them, the token is still valid until expiry. Solution: short-lived access tokens (~15 min) + refresh tokens.
- **Long-lived tokens are dangerous.** Stolen = full access until expiry.
- **Always use HTTPS** — JWT in plain HTTP is broadcast in plain text.
- **Store secret in a vault**, not in `appsettings.json` checked into Git.

- Access token (short-lived, ~15 min) for API calls.
- Refresh token (longer-lived, ~7 days, stored server-side) to get new access tokens.
- Logout → invalidate the refresh token in DB.

- Putting sensitive data (email, phone, full name) in JWT claims — visible in client storage.
- Using long-lived tokens with no refresh mechanism — bad UX or bad security, pick one.
- Storing tokens in `localStorage` — vulnerable to XSS. Prefer httpOnly cookies for browser apps.
- Validating only signature, not expiration / issuer / audience.

```
// Program.cs — JWT setup
builder.Services
    .AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(o =>
    {
        var jwt = builder.Configuration.GetSection("Jwt");
        o.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = true,
            ValidateAudience = true,
            ValidateLifetime = true,
            ValidateIssuerSigningKey = true,
            ValidIssuer = jwt["Issuer"],
            ValidAudience = jwt["Audience"],
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(jwt["Key"]!))
        };
    });

builder.Services.AddAuthorization();

app.UseAuthentication();
app.UseAuthorization();

// Issuing a token
public string Generate(User user)
{
    var claims = new[]
    {
        new Claim(ClaimTypes.NameIdentifier, user.Id.ToString()),
        new Claim(ClaimTypes.Role, user.Role),
        new Claim("tenant", user.TenantId.ToString())
    };

    var key = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(_settings.Key));
    var creds = new SigningCredentials(key, SecurityAlgorithms.HmacSha256);

    var token = new JwtSecurityToken(
        issuer: _settings.Issuer,
        audience: _settings.Audience,
        claims: claims,
        expires: DateTime.UtcNow.AddMinutes(15),
        signingCredentials: creds);

    return new JwtSecurityTokenHandler().WriteToken(token);
}
```

- *Why use refresh tokens?* — Because access tokens are short-lived (good for security). Refresh tokens let the client get a new access token without making the user log in again. Refresh tokens **can** be revoked server-side.
- *How would you handle JWT logout?* — JWTs themselves can't be invalidated. Options: (1) short expiry + revoke refresh token, (2) maintain a token blacklist (defeats statelessness), (3) rotate the signing key (drastic, logs everyone out).

Q2: What's the difference between Authentication and Authorization, and how do roles, policies, and claims fit in?

Beginners use these terms interchangeably; seniors don't.

- **Authentication** = "who are you?" → verifying identity (login, JWT, cookie).

- **Authorization** = "what can you do?" → checking permissions on identified users.

Order in ASP.NET Core: **Authentication first, then Authorization.** That's why `app.UseAuthentication()` comes before `app.UseAuthorization()`.

1. Roles — simplest. User has roles like `Admin`, `Manager`. `[Authorize(Roles = "Admin")]`.

- Pros: easy.
- Cons: rigid; doesn't scale beyond a few roles.

2. Policies — flexible, named rules. `[Authorize(Policy = "MustBeAdult")]`.

- Pros: combine multiple requirements; reusable; easy to test.
- Cons: slightly more setup.

3. Claims — fine-grained data on the user (`tenant`, `permission:invoice.read`).

- Pros: most flexible; works well with JWTs.
- Cons: can become unmanageable if not organized.

My real-world rule: start with roles, move to **policies + claims** when permissions get complex (multi-tenant, fine-grained features). Mixing all three is fine.

Resource-based authorization is another senior-level concept: when "can the user edit this **specific** order?" depends on data (e.g., they can only edit their own). Use `IAuthorizationService` to check at the resource level — not just the endpoint.

- Hardcoding role checks in controllers (`if (user.IsInRole("Admin"))`) — duplicates logic. Use `[Authorize]`.
- Putting all permission logic on the frontend. **Always** enforce on the backend — the frontend is just a UX hint.
- Forgetting that JWT roles need to be extracted as `ClaimTypes.Role` claims for `[Authorize(Roles = "...")]` to work.
- Confusing `[Authorize]` (require auth) with `[AllowAnonymous]` (skip auth).

```
// Roles
[Authorize(Roles = "Admin,Manager")]
[HttpDelete("{id}")]
public IActionResult Delete(int id) => NoContent();

// Policy setup
builder.Services.AddAuthorization(opts =>
{
    opts.AddPolicy("MustBeAdult", p => p.RequireClaim("age", "18", "19", "20")); // example
    opts.AddPolicy("CanEditInvoices", p => p.RequireClaim("permission", "invoice.edit"));
});

[Authorize(Policy = "CanEditInvoices")]
[HttpPut("{id}")]
public IActionResult Update(int id, InvoiceDto dto) => Ok();

// Resource-based — only the order owner can update
public async Task<IActionResult> Update(int id)
{
    var order = await _db.Orders.FindAsync(id);
    if (order is null) return NotFound();

    var result = await _authz.AuthorizeAsync(User, order, "OrderOwnerPolicy");
    if (!result.Succeeded) return Forbid();

    // proceed
    return NoContent();
}
```

- *When would you choose claims over roles?* — When permissions are **fine-grained** and **dynamic** — for example, in multi-tenant apps where each user has different permissions per tenant. Roles are too coarse.
- *How do you authorize based on the resource itself, not just the endpoint?* — Use **resource-based authorization** with `IAuthorizationService.AuthorizeAsync(user, resource, policy)`. The handler can inspect the resource (e.g., check `order.UserId == user.Id`).

TOPIC 14: REST API BEST PRACTICES

Q1: What makes an API truly RESTful and what are your best practices? (IMPORTANT)

A: REST is an **architectural style**, not a strict protocol. The core idea: model your API around **resources**, use HTTP correctly, and stay **stateless**.

1. **Resources, not actions.** URLs name **things** (`/api/orders/123`), not verbs (`/api/getOrder?id=123`).
2. **HTTP verbs define the action.** `GET` to read, `POST` to create, `PUT`/`PATCH` to update, `DELETE` to remove.
3. **Stateless.** No session on the server. Every request carries its own auth and context.
4. **Standard HTTP status codes.**
5. **Representations** (typically JSON) decouple the resource from how it's serialized.
6. **Cacheability** via response headers (`Cache-Control`, `ETag`).

- **Pluralize resource names** — `/users`, `/orders`. Consistent.
- **Use nouns, not verbs.** `POST /api/users` (not `POST /api/createUser`).
- **Hierarchical relationships** — `/users/5/orders`, `/orders/12/items`.
- **Use query strings for filters/pagination** — `/api/users?status=active&page=2&size=20`.
- **Return proper status codes** (covered in the Web API topic).
- **Use Problem Details (RFC 7807)** for error responses — standard, machine-readable.
- **Pagination** by default — never return unbounded lists.
- **Versioning** from day one — `/api/v1/users`.
- **Idempotency** for `PUT` and `DELETE` — calling twice should have the same effect as once. For `POST`, use **idempotency keys** for safety.
- **HATEOAS** is theoretically pure REST, but pragmatically rarely worth it.
- **HTTPS everywhere.**
- **Rate limiting** — built in since .NET 7.
- **Document with OpenAPI/Swagger.**
- **Consistent naming convention** for fields (camelCase JSON is standard).

Real-world example: I designed an API where `POST /orders` returned the new resource with status `201` and a `Location: /api/orders/123` header. Validation failures returned `400` with Problem Details, including which fields failed. Listing was paginated by default with sane limits. Rate limit was 100 req/min per user. Frontend devs had **zero** custom handling per endpoint — everything followed the same pattern.

- "RESTful" but with verbs in URLs (`/getUserById`, `/saveOrder`).
- Returning `200 OK` with `success: false` — clients can't react properly.
- No pagination → first time the table has 100K rows, the API dies.
- Inconsistent naming — `userId` in one endpoint, `user_id` in another.
- Returning entire DB rows in responses — exposes internal columns, breaks compatibility.

```
// Idempotent PUT
[HttpPut("{id:int}")]
public async Task<IActionResult> Update(int id, UpdateUserDto dto, CancellationToken ct)
{
    if (!await _users.ExistsAsync(id, ct)) return NotFound();
    await _users.UpdateAsync(id, dto, ct);
    return NoContent();    // 204 — success, nothing to return
}

// Pagination by default
[HttpGet]
public async Task<ActionResult<PagedResult<UserDto>>> List(
    [FromQuery] int page = 1,
    [FromQuery] int size = 20,
    CancellationToken ct = default)
{
    if (size > 100) size = 100;    // hard cap
    var data = await _users.GetPageAsync(page, size, ct);
    return Ok(data);
}
```

- *What is HATEOAS and is it worth implementing?* — HATEOAS = "Hypermedia As The Engine Of Application State" — responses include links to related actions. Pure REST requires it, but in practice almost no one uses it. Cost > benefit for typical APIs.
- *How do you ensure backward compatibility when evolving an API?* — Add new fields (don't remove); keep old endpoints alongside new ones; version when you make breaking changes; communicate deprecation timelines.

TOPIC 15: CACHING

Q1: What caching strategies do you use in real .NET applications?

A: Caching is about **trading freshness for speed and reduced load**. The senior trick is knowing **what to cache, where, and for how long** — not just "I added a cache."

1. **In-process cache** (`IMemoryCache`) — fastest, in app memory. Lost on restart, not shared across instances.
2. **Distributed cache** (`IDistributedCache` — typically **Redis**) — shared across all app instances. Slower than in-memory but consistent.
3. **HTTP / Response cache** — `Cache-Control` and `ETag` headers, plus output caching middleware (.NET 7+).
4. **CDN / Edge cache** — for static assets and public read endpoints.
5. **Database query cache** — EF Core has limited support; second-level caches like `EFCoreSecondLevelCacheInterceptor` exist but used carefully.

- **Cache-aside** (most common) — check cache, miss → read DB → populate cache. App controls everything.
- **Read-through / Write-through** — cache library handles fetch/update transparently.
- **Refresh-ahead** — proactively refresh near expiry so users don't see misses.

- Reference data that rarely changes (countries, currencies, role definitions).
- Expensive read queries (dashboards, reports).
- External API responses (with respect to their TTL).
- User sessions / JWT validation results in some cases.

- User-specific data without proper key isolation.
- Frequently changing data (live stock, balances).
- Anything where staleness causes correctness bugs.

Real-world example: I had an endpoint `/api/products` that served the homepage. 10K hits/min, hitting EF Core every time. Response time ~250 ms. Added 5-min in-memory cache → response time dropped to **2 ms** with 99% hit rate. DB CPU dropped 80%.

- Caching everything → stale data bugs and memory bloat.
- No cache eviction policy → memory leak.
- Caching per-user data with shared keys → users see each other's data (security incident).
- **Cache stampede** — when a popular key expires, 1000 requests miss the cache simultaneously and hammer the DB. Solution: **lock around regeneration**, use SWR (stale-while-revalidate), or use `HybridCache` (.NET 9 / preview in 8).

```
// Cache-aside with IMemoryCache
public class ProductService
{
    private readonly IMemoryCache _cache;
    private readonly IProductRepository _repo;

    public ProductService(IMemoryCache cache, IProductRepository repo)
    {
        _cache = cache;
        _repo = repo;
    }

    public Task<List<ProductDto>> GetActiveAsync(CancellationToken ct)
        => _cache.GetOrCreateAsync("products:active", async entry =>
        {
            entry.AbsoluteExpirationRelativeToNow = TimeSpan.FromMinutes(5);
            entry.SlidingExpiration = TimeSpan.FromMinutes(2);
            return await _repo.GetActiveAsync(ct);
        })!;
}
```

- *What is cache stampede and how do you prevent it?* — When a popular key expires, many requests miss simultaneously and all hit the source. Prevention: locking around the regeneration (e.g., `SemaphoreSlim`), serving stale-while-revalidate, or using `HybridCache` which handles this for you.
- *How do you handle cache invalidation across multiple app instances?* — Use a **distributed cache (Redis)** with a **pub/sub channel** to notify other instances when a key changes. Or rely on TTL-based invalidation if eventual consistency is acceptable.

Q2: IMemoryCache vs IDistributedCache (Redis) — when do you use which?

A: Both serve cached data, but they live in **different places** and solve different problems.

Feature	`IMemoryCache`	`IDistributedCache` (Redis)
Location	In-process (app memory)	External server (Redis, SQL, etc.)
Speed	Sub-millisecond	~1–5 ms (network)
Shared across instances	❌ No	✅ Yes
Survives app restart	❌ No	✅ Yes
Type support	Any object	Byte arrays / strings (manual serialization)
Use when	Single-instance, hot data	Multi-instance, shared state

- **Single instance / dev / small app** → `IMemoryCache`. Simple, fast, no extra infra.
- **Multiple instances behind a load balancer** → `IDistributedCache` with **Redis**. Mandatory if cache consistency matters across pods.
- **Often the right answer is both** — `HybridCache` (preview in .NET 8, GA later) gives you in-memory L1 + distributed L2 with proper stampede protection. Best of both worlds.

- Sub-millisecond reads on the same network.
- Pub/Sub for cross-instance invalidation.
- Rich data types (lists, sets, sorted sets, streams).
- Cluster mode for horizontal scale.

- Using `IMemoryCache` in a multi-instance deployment → instances have inconsistent caches → bizarre user-facing bugs.
- Using `IDistributedCache` for tiny, hot data that fits in process memory → unnecessary network calls.
- Not setting expiration → memory leak (in-process) or slow Redis growth.
- Not handling Redis being down → app crashes. Always have **fallback behavior**.

Real-world example: an app started single-instance, used `IMemoryCache`. Scaled to 4 pods → users complained about seeing stale data depending on which pod served them. Switched to Redis distributed cache. Problem solved, but added ~3 ms per cache hit. Worth it for consistency.

```
// IMemoryCache
builder.Services.AddMemoryCache();

// IDistributedCache (Redis)
builder.Services.AddStackExchangeRedisCache(o =>
{
    o.Configuration = builder.Configuration.GetConnectionString("Redis");
});

public class CountryService
{
    private readonly IDistributedCache _cache;
    public CountryService(IDistributedCache cache) => _cache = cache;

    public async Task<List<Country>> GetAllAsync(CancellationToken ct)
    {
        const string key = "countries:all";
        var cached = await _cache.GetStringAsync(key, ct);
        if (cached is not null)
            return JsonSerializer.Deserialize<List<Country>>(cached)!;

        var fresh = await LoadFromDbAsync(ct);
        await _cache.SetStringAsync(key, JsonSerializer.Serialize(fresh),
            new DistributedCacheEntryOptions { AbsoluteExpirationRelativeToNow = TimeSpan.FromHours(24) }, ct);

        return fresh;
    }

    private Task<List<Country>> LoadFromDbAsync(CancellationToken ct) => Task.FromResult(new List<Country>());
}
```

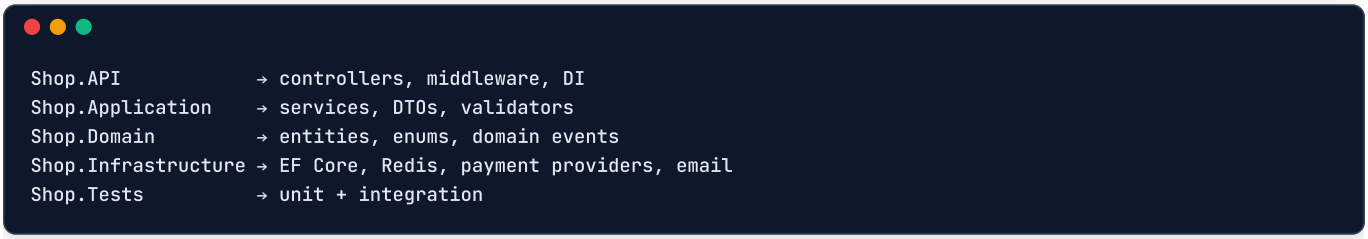
- *What is HybridCache (.NET 8/9)?* — A new caching abstraction that gives you in-process L1 + distributed L2 with built-in **stampede protection**, tag-based invalidation, and serialization. Ideal modern choice.
- *How do you handle cache failures gracefully?* — Wrap cache calls in try/catch and **fall back to the source** on failure. Cache should be an optimization, never a hard dependency.

TOPIC 16: BASIC SYSTEM DESIGN THINKING

Q1: How would you design a small e-commerce backend in .NET? (IMPORTANT)

A: This is the kind of question that separates candidates who've **built** systems from those who've only learned about them. I'll walk through how I'd actually think about it.

I'd ask: how many users? Single region or global? Read-heavy or write-heavy? What features (catalog, cart, checkout, orders, payments)? For a small-scale system, let's assume ~100K users, single region, standard catalog + checkout.



- `AuthService` — JWT issue + refresh tokens.
- `ProductService` — catalog browsing, cached aggressively (5-min TTL on lists, ETag on details).
- `CartService` — Redis-backed (cart is ephemeral, doesn't need DB).
- `OrderService` — DB-backed; uses **idempotency key** on POST to prevent duplicate orders if the user double-clicks.
- `PaymentService` — `IPaymentProvider` abstraction; Stripe for now.
- `InventoryService` — handles stock decrement; uses **DB transaction** with `FOR UPDATE` / row-locking to avoid overselling.
- `NotificationService` — sends emails via background queue; never blocks the request.

- **Logging:** Serilog → JSON → Seq/ELK with correlation IDs.
- **Caching:** Redis for catalog, in-memory for static config.
- **Auth:** JWT with refresh tokens.
- **Rate limiting:** built-in middleware (.NET 7+) — protect login & checkout.
- **Validation:** FluentValidation in service layer.
- **Background jobs:** Hangfire or a Hosted Service for email sending, report generation.
- **Health checks:** `/health` endpoint for k8s readiness.
- **Migrations:** EF Core migrations applied via CI/CD.

- **Monolith vs microservices** — for 100K users, a well-structured **modular monolith** is easier to operate and just as scalable. Microservices later if a specific module needs independent scaling.
- **SQL vs NoSQL** — SQL for orders/payments (ACID), Redis for cart/cache. Don't reach for MongoDB just because it's cool.
- **Sync vs async checkout** — accept the order synchronously (you need immediate confirmation), but **process payment confirmation, inventory deduction, and email notifications asynchronously** to keep response time fast.

- **Retry with exponential backoff** for payment & email.
- **Circuit breaker** (Polly) on external dependencies.
- **Idempotency keys** on POSTs.
- **Optimistic concurrency** on inventory updates.

- Jumping straight to microservices for a 100K-user app — over-engineering.

- Not asking clarifying questions — interviewers want to see how you scope.
- Ignoring failure modes (what if Stripe is down? what if Redis is down?).
- Treating the DB as infinitely fast — a senior thinks about query patterns and indexes from day one.

```
// Example: idempotent order creation
[HttpPost]
public async Task<IActionResult> CreateOrder(
    OrderDto dto,
    [FromHeader(Name = "Idempotency-Key")] string idempotencyKey,
    CancellationToken ct)
{
    if (string.IsNullOrEmpty(idempotencyKey))
        return BadRequest("Idempotency-Key header required");

    var existing = await _orders.GetByIdempotencyKeyAsync(idempotencyKey, ct);
    if (existing is not null) return Ok(existing); // safe replay

    var order = await _orders.CreateAsync(dto, idempotencyKey, ct);
    return CreatedAtAction(nameof(Get), new { id = order.Id }, order);
}
```

- *How would you handle high traffic spikes during a sale?* — Pre-warm Redis cache, scale API pods horizontally, queue payment processing, use rate limiting per user, consider a CDN for product images, and have a static "fallback" page if checkout DB is overwhelmed.
- *How would you split this into microservices later?* — Identify **bounded contexts** — Identity, Catalog, Cart, Orders, Payments, Notifications. Extract one at a time, starting with the one that needs **independent scaling** or has a different team owning it.

TOPIC 17: SQL BASICS

Q1: Explain the different SQL Joins. (IMPORTANT)

Joins combine rows from two or more tables based on related columns. As a backend dev, you should know these cold — most real-world bugs in reports come from picking the wrong join.

Join	Returns
INNER JOIN	Only rows that **match in both** tables
LEFT JOIN	All rows from the **left** + matching from right (NULL if none)
RIGHT JOIN	All rows from the **right** + matching from left (rarely used)
FULL OUTER JOIN	All rows from both, NULL where no match
CROSS JOIN	Cartesian product — every combination
SELF JOIN	Table joined with itself (e.g., employee→manager)

```
-- Looks like LEFT JOIN, but silently behaves like INNER JOIN
SELECT u.Name, o.Id
FROM Users u
LEFT JOIN Orders o ON u.Id = o.UserId
WHERE o.Status = 'Completed';   -- 💀 filters out users without orders
```

The fix — move the condition to the `ON` clause:

```
SELECT u.Name, o.Id
FROM Users u
LEFT JOIN Orders o ON u.Id = o.UserId AND o.Status = 'Completed';
```

This is an extremely common bug. Always remember: filtering on a right-side column in `WHERE` of a `LEFT JOIN` quietly turns it into an `INNER JOIN`.

Real-world example: I built a "users without recent orders" report. First version used `LEFT JOIN` + `WHERE o.CreatedAt > ...` and missed users with **no orders at all** (the whole point). Fixed by moving the date filter into the `ON` clause and checking `WHERE o.Id IS NULL` for users with no orders in the period.

Tip freshers don't know: `EXISTS` and `NOT EXISTS` are usually **faster than equivalent JOIN + DISTINCT** patterns, because the optimizer can short-circuit.

```
-- ✅ Faster than LEFT JOIN + IS NULL
SELECT u.* FROM Users u
WHERE NOT EXISTS (SELECT 1 FROM Orders o WHERE o.UserId = u.Id);
```

```
-- INNER JOIN — only users with orders
SELECT u.Name, o.Total
FROM Users u
INNER JOIN Orders o ON u.Id = o.UserId;

-- LEFT JOIN — all users, with order count (0 if none)
SELECT u.Name, COUNT(o.Id) AS OrderCount
FROM Users u
LEFT JOIN Orders o ON u.Id = o.UserId
GROUP BY u.Name;

-- SELF JOIN
SELECT e.Name AS Employee, m.Name AS Manager
FROM Employees e
LEFT JOIN Employees m ON e.ManagerId = m.Id;
```

- *What's the difference between `WHERE` and `HAVING`?* — `WHERE` filters **rows** before grouping; `HAVING` filters **groups** after `GROUP BY`. You can't use aggregates (`COUNT`, `SUM`) in `WHERE` — use `HAVING`.
- *Why might `EXISTS` be faster than `IN` for large subqueries?* — `EXISTS` short-circuits at the first match; the optimizer can use a semi-join. `IN` materializes the entire subquery first. For big subqueries, `EXISTS` wins.

Q2: What is Database Normalization?

A: Normalization organizes tables to **reduce redundancy** (same data repeated in many places) and **avoid anomalies** (insert/update/delete causing inconsistency).

Normal Form	Rule (in plain words)
1NF	Atomic columns — no comma-separated lists, no repeating groups
2NF	1NF + every non-key column depends on the **whole** primary key
3NF	2NF + non-key columns don't depend on **other non-key columns**
BCNF	Stricter 3NF — every determinant is a candidate key

In real apps, most teams go to **3NF** and selectively **denormalize** for performance.

Real-world example: A junior added an `Orders` table with `CustomerName`, `CustomerEmail`, `CustomerPhone` on **every order row**. When customers updated their phone, the new number only showed on new orders. Old orders kept the stale data. We split into `Customers` (one row each) + `Orders` (with `CustomerId` FK). Now updating a phone is one row.

- **Read-heavy reporting** — calculating `OrderTotal` from line items every time is slow; store the total on the order row.
- **Audit trails** — sometimes you **want** to capture the snapshot at the moment (e.g., `ProductPriceAtOrderTime`).
- **Multi-tenant** — sometimes denormalize tenant data to avoid massive joins.

- Over-normalizing — every report becomes a 7-table join, performance dies.
- Under-normalizing — duplicate data, update anomalies, eventual chaos.
- Forgetting that **denormalization is a trade-off** — you trade write complexity for read speed. Document the choice.

```sql
-- ❌ Not normalized — customer info repeated
CREATE TABLE Orders (
    Id INT PRIMARY KEY,
    CustomerName NVARCHAR(100),
    CustomerEmail NVARCHAR(100),
    Amount DECIMAL
);

-- ✅ Normalized
CREATE TABLE Customers (
    Id INT PRIMARY KEY,
    Name NVARCHAR(100),
    Email NVARCHAR(100)
);
CREATE TABLE Orders (
    Id INT PRIMARY KEY,
    CustomerId INT REFERENCES Customers(Id),
    Amount DECIMAL,
    CreatedAt DATETIME2 NOT NULL
);
```

- *When would you intentionally break normalization?* — For **read performance** in reporting tables, **historical snapshots** (capture data at a moment in time), or **denormalized read models** in CQRS-style architectures.
- *What's the difference between a primary key and a unique constraint?* — A **primary key** uniquely identifies a row, can't be null, and there's only one per table. A **unique constraint** also enforces uniqueness but allows nulls and you can have many per table.

Q3: What is Indexing and how does it affect performance?

A: An index is a **separate data structure** (typically B-tree) that lets the DB find rows quickly, the same way a book index helps you find a topic without reading every page.

Without index: the DB does a **table scan** — reads every row. O(N). **With index:** B-tree lookup. O(log N) — for a million rows, **~20 reads vs 1,000,000**.

- **Clustered index** — defines the **physical order** of rows in the table. **One per table.** In SQL Server it's usually the primary key.
- **Non-clustered index** — separate structure pointing to rows. **Multiple per table.**
- **Composite index** — covers multiple columns; column **order matters**.
- **Covering index** — includes all columns needed by a query so the DB doesn't touch the table.

Real-world example: I had a query `SELECT * FROM Orders WHERE CustomerId = @id ORDER BY CreatedAt DESC` taking 4 seconds on 5M rows. Added a composite index on `(CustomerId, CreatedAt DESC)` → dropped to **8 ms**. 500× faster, no code change.

- **WHERE / JOIN columns** that are filtered or joined frequently.
- **Foreign keys** (EF Core doesn't always create these — check your migrations).
- **ORDER BY columns** when used with WHERE.
- Columns in **selective WHERE clauses** (high distinct values).

- **Columns rarely used in WHERE.** Wasted storage + slower writes.
- **Tiny tables (< few thousand rows).** Scan is faster.
- **Columns with low cardinality** (e.g., `IsActive` boolean) — index isn't selective enough.

> *Indexes speed up **reads** but slow down **writes** (every INSERT/UPDATE/DELETE must update every relevant index). Don't add indexes blindly — measure first.*

- Adding indexes "to be safe" without measuring — slows writes for no read benefit.
- Wrong column order in composite indexes — `(B, A)` doesn't help queries that filter by `A` alone.
- No `INCLUDE` columns when needed — the query still has to do a key lookup back to the table.
- Forgetting to **rebuild fragmented indexes** periodically (less an issue in modern SQL Server with auto-tuning).

```sql
-- Composite index — column order matters!
CREATE INDEX IX_Orders_CustomerId_CreatedAt
ON Orders (CustomerId, CreatedAt DESC);

-- Covering index — INCLUDE non-key columns the query needs
CREATE INDEX IX_Orders_Status
ON Orders (Status)
INCLUDE (Total, CreatedAt);

-- Find missing indexes (SQL Server)
SELECT * FROM sys.dm_db_missing_index_details;
```

- *Why does the column order in a composite index matter?* — Indexes are sorted by the **leftmost column first**. An index on `(A, B)` helps queries filtering by `A`, or `A AND B`, but **not** queries filtering by `B` alone — you'd need a separate index.
- *What is index fragmentation?* — Over time, inserts/updates leave gaps and out-of-order pages in the index, slowing scans. Periodic rebuild or reorganize fixes it; modern SQL Server has auto-tuning that handles most cases.

TOPIC 18: PROJECT STRUCTURE & CLEAN ARCHITECTURE

Q1: How do you structure a real-world ASP.NET Core project (Clean Architecture basics)?

(IMPORTANT)

A: A clean structure makes the difference between a project you can **change confidently** and one where every PR risks breaking 5 other things. For most small-to-medium ASP.NET Core projects I follow a **lightweight Clean Architecture** with 4 layers.

```
Shop.API            → Controllers, Program.cs, middleware, DI setup
Shop.Application    → Services, DTOs, use cases, validators, interfaces (IRepo)
Shop.Domain         → Entities, value objects, enums, domain events, business rules
Shop.Infrastructure → EF Core (DbContext, migrations, repos), 3rd-party integrations (Stripe, SendGrid)
Shop.Tests          → Unit + integration tests
```

> *Dependencies always point **inward**. The Domain depends on **nothing**. The Application depends only on Domain. Infrastructure and API depend on Application/Domain (never the other way around).*

This means you can **rip out** EF Core or Stripe without touching business logic. You can **unit test** the Application layer without spinning up a DB.

```
[HTTP request]
    → Controller (API)            ← validates, maps DTO
    → Service (Application)       ← business rules
    → Repository (Infrastructure) ← EF Core / external API
    → DB
```

- **Domain** — `Order`, `OrderItem`, `Money`, `OrderStatus`. Business rules: `Order.AddItem(...)`, `Order.MarkPaid(...)`. **No EF Core annotations, no DI, no** `using Microsoft.*`.
- **Application** — `IOrderRepository` (interface), `CreateOrderService`, `OrderDto`. Use cases live here.
- **Infrastructure** — `OrderRepository : IOrderRepository`, `AppDbContext`, `StripePaymentProvider`. The "how."
- **API** — `OrdersController`, middleware, `Program.cs`.

- **Testable** — services have no DB dependency; tests run in seconds.
- **Swappable** — change SQL Server to PostgreSQL by changing only Infrastructure.
- **Onboarding-friendly** — new devs see the layers and immediately know where to look.

- For a **tiny CRUD API** (< 5 endpoints), it's overkill. A single project is fine.
- The **abstraction has cost** — more interfaces, more files. Apply when the project's complexity justifies it.

Real-world example: the Web API I built last year follows this exactly. Controllers are 20–40 lines (validate input, call service, return result). All business rules live in services. We swapped EF Core for Dapper in the reporting module by changing **only the Infrastructure layer** — zero impact on services or controllers.

- Putting EF Core entities in Domain. Then Domain depends on EF — defeats the architecture.
- Putting `DbContext` directly in controllers. Skips two layers, breaks testability.
- Putting business logic in repositories. Repositories should be **dumb data access**.

- Creating `IService` for every concrete service "for DI." Only abstract when you need flexibility — most internal services don't.

```
// Domain — pure business
public class Order
{
    public int Id { get; private set; }
    public OrderStatus Status { get; private set; } = OrderStatus.Pending;
    private readonly List<OrderItem> _items = new();
    public IReadOnlyList<OrderItem> Items => _items;

    public void AddItem(int productId, int qty, decimal price)
    {
        if (Status != OrderStatus.Pending) throw new DomainException("Cannot modify confirmed order");
        _items.Add(new OrderItem(productId, qty, price));
    }

    public void Confirm()
    {
        if (_items.Count == 0) throw new DomainException("Cannot confirm empty order");
        Status = OrderStatus.Confirmed;
    }
}

// Application — use case
public class CreateOrderService(IOrderRepository repo, IUnitOfWork uow)
{
    public async Task<int> CreateAsync(CreateOrderDto dto, CancellationToken ct)
    {
        var order = new Order();
        foreach (var item in dto.Items)
            order.AddItem(item.ProductId, item.Qty, item.Price);
        order.Confirm();

        await repo.AddAsync(order, ct);
        await uow.SaveChangesAsync(ct);
        return order.Id;
    }
}

// API — thin controller
[ApiController]
[Route("api/[controller]")]
public class OrdersController(CreateOrderService service) : ControllerBase
{
    [HttpPost]
    public async Task<IActionResult> Create(CreateOrderDto dto, CancellationToken ct)
    {
        var id = await service.CreateAsync(dto, ct);
        return CreatedAtAction(nameof(Get), new { id }, null);
    }

    [HttpGet("{id:int}")]
    public IActionResult Get(int id) => Ok();
}
```

- *Do you always use the Repository pattern with EF Core?* — Honest answer: not always. EF Core's `DbSet<T>` is already a repository and unit of work. Adding `IRepository<T>` on top is often **redundant abstraction**. I add repositories only when I need to **isolate domain from EF**, **share complex queries**, or **swap the data source** later.
- *What's the difference between Clean Architecture and Onion Architecture?* — They're nearly identical in practice. Both enforce the **dependency rule** (inward-only). Onion uses concentric circles; Clean uses layers with the same idea. Pick whichever vocabulary your team prefers.

QUICK REVISION CHEAT SHEET

Topic	Key Points
Value vs Ref Types	Value = stack + copy; Reference = heap + shared; `string` is reference but immutable. Boxing = perf killer.
var / dynamic / object	`var` = compile-time; `object` needs cast; `dynamic` = runtime, slow, no IntelliSense.
string vs StringBuilder	StringBuilder for **loops / many concatenations**. `string.Join` for collections.
OOP Pillars	Encapsulation (hide data), Abstraction (hide complexity), Inheritance, Polymorphism.
Abstract vs Interface	Interface = capability; Abstract = shared code. **Default to interfaces.**
virtual / override / new	`override` for polymorphism; `new` silently breaks it.
SOLID	SRP, OCP, LSP, ISP, DIP — most important: **SRP and DIP**.
.NET 6/7/8	Minimal APIs, Rate Limiter, Native AOT, **Keyed DI**, `IExceptionHandler`, `TimeProvider`.
Middleware Order	Exception → HTTPS → Static → Routing → CORS → **Auth** → **Authz** → Endpoints.
DI Lifetimes	Singleton (app), Scoped (request — DbContext), Transient (per resolve). Avoid **captive deps**.
EF Core	Use `AsNoTracking` for reads. Watch N+1 — fix with `Include` or projection. `DbContext` is Scoped.
LINQ	`IQueryable` = SQL; `IEnumerable` = memory. Stay in `IQueryable` until you materialize.
Async	"Async all the way." Avoid `.Result` / `.Wait()`. Use `CancellationToken`.
Exception Handling	Catch specific. Use `throw;`, never `throw ex;`. Global handler + structured logging.
Status Codes	200/201/204, 400/401/403/404/409/422/429, 500/503. **401 ≠ 403.**
Filters vs Middleware	Middleware for cross-cutting; filters when you need MVC context.
Versioning	URL path versioning is most practical. Asp.Versioning library. Plan from day one.
JWT	Stateless, signed (not encrypted). Short-lived access + refresh tokens. HTTPS only.
Auth	Authentication = who. Authorization = what allowed. Roles → Policies → Claims as complexity grows.
REST	Nouns not verbs. Standard verbs/codes. Pagination by default. Idempotency keys for POST.
Caching	In-memory for single-instance; Redis for multi-instance. Watch cache stampede.
System Design	Start simple. Modular monolith > microservices for small apps. Idempotency. Resilience.
SQL Joins	INNER, LEFT, RIGHT, FULL. Filter on outer-side column → use `ON`, not `WHERE`.
Normalization	1NF → 2NF → 3NF. Denormalize for read perf, deliberately.
Indexes	Speed reads, slow writes. Composite index — column order matters.
Project Structure	API → Application → Domain → Infrastructure. **Dependencies point inward.**

INTERVIEW TIPS

1. **Start with a direct, confident answer** — define the concept in one sentence, then go deep.
2. **Always connect to a real project** — "In my project, we used this when..." beats any textbook line.
3. **Show decision-making, not just knowledge** — explain **why** you chose X over Y, and **when NOT to** use it.
4. **Mention trade-offs** — every senior answer includes a trade-off. "It's faster but uses more memory." "Easier to test but harder to scale."
5. **Use the right keywords** — `AsNoTracking`, `Scoped`, `IExceptionHandler`, `IQueryable`, `idempotency`, `cache stampede`, `correlation ID`. Interviewers listen for these.
6. **Be honest** — "I haven't used X in production but I understand the concept" is much better than bluffing. Bluffs collapse on the second follow-up.
7. **Be ready to write code** — small things like a controller, LINQ query, DI registration, custom middleware, or a `try/catch` should flow without thinking.
8. **Ask clarifying questions** in design / scenario problems — interviewers want to see how you scope.
9. **Talk through your thinking** — silence during a problem feels much worse than thinking out loud.
10. **Stay calm with "I don't know"** — pick the closest concept you do know and reason from there. Curiosity beats confidence.

Generated for .NET Interview Preparation | 0 – 2.5 Years Experience | Senior-Style Answers | Last Updated: April 2026